

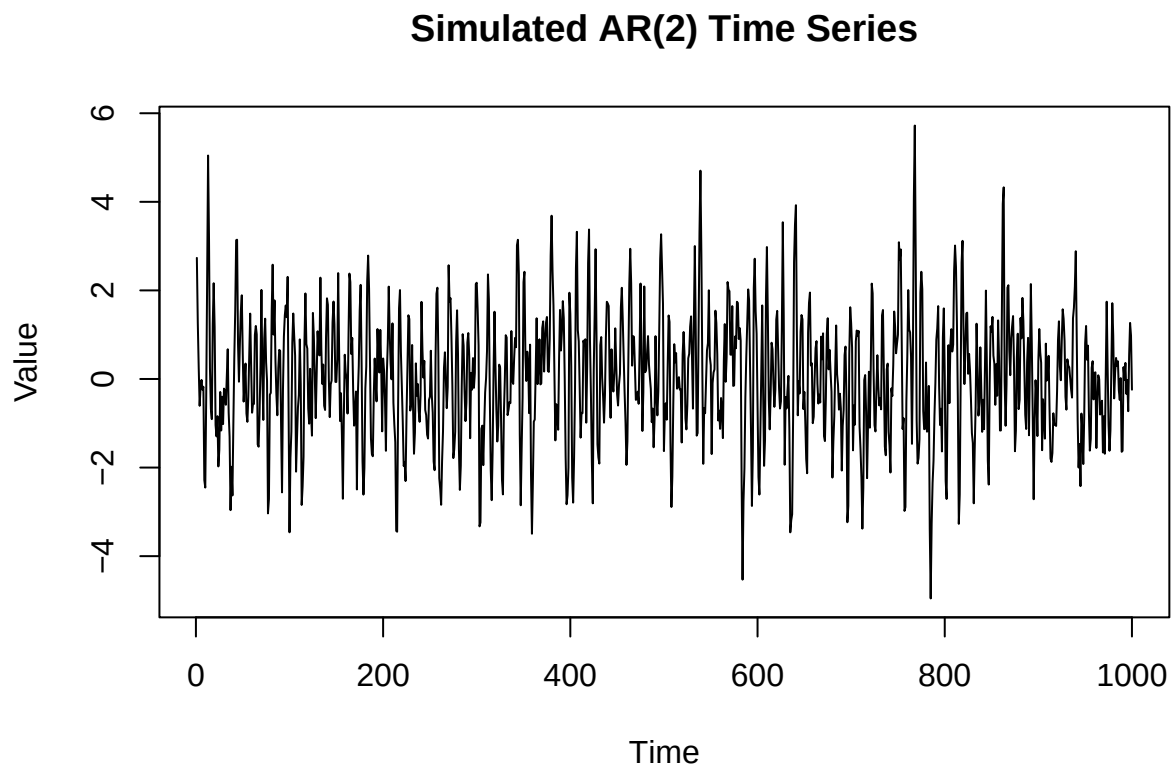
Practical 3 - AR(p) Models

Practical 3a AR(p) Models

1. Simulating AR(p) models

Generate an AR(2) model:

```
n<-1000
x<-arima.sim(list(ar = c(0.8897, -0.4858), sd = sqrt(0.1796)),n)
plot(x,type="l",main="Simulated AR(2) Time Series", ylab="Value", xlab="Time")
```

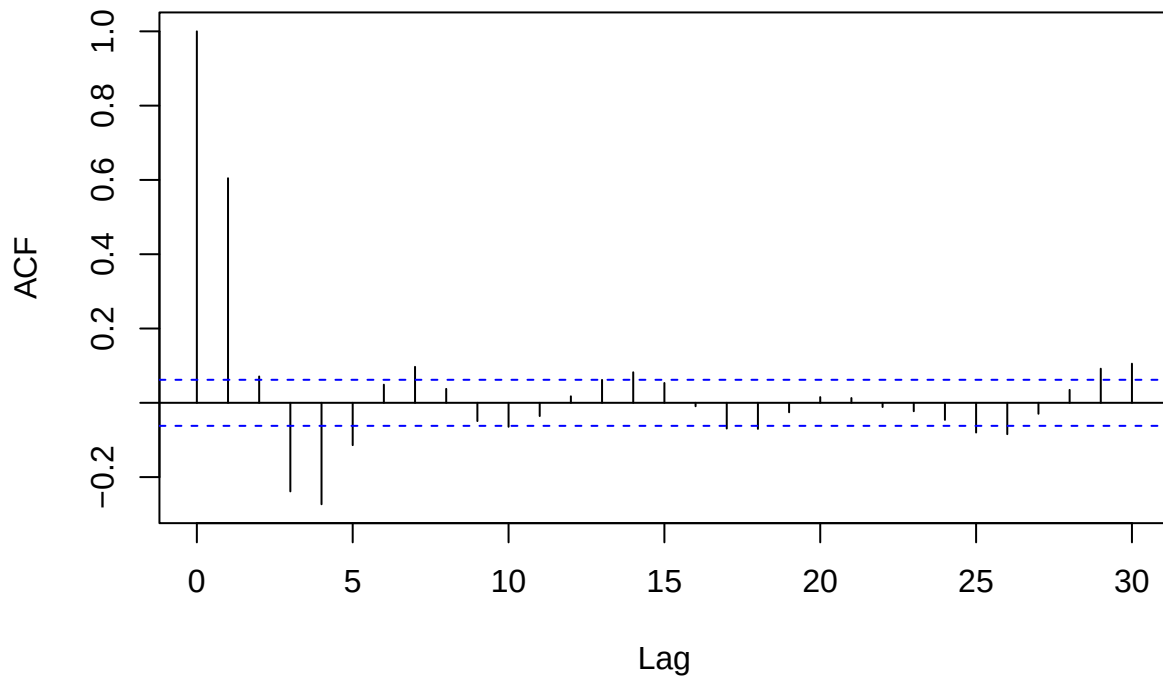


The simulated series shows a stochastic behavior with no clear trend or seasonality, as expected from an AR(p) process. The values fluctuate around a constant mean, suggesting stationarity.

Compute its autocorrelation function:

```
acf(x, main="ACF of Simulated AR(2)")
```

ACF of Simulated AR(2)



The autocorrelation function shows a gradual decay (kind of exponential), which is characteristic of AR processes. This confirms that the data was generated from an AR model.

Repeat for different n , number of parameters and different p of the $AR(p)$.

First I create a function to build the $AR(p)$ model, so that I can easily vary n , p and the parameters ϕ :

```
simulate_ar <- function(n, phi, sigma2 = 1) {  
  
  x <- arima.sim(  
    model = list(ar = phi),  
    n = n,  
    sd = sqrt(sigma2)  
  )  
  
  old_par <- par(no.readonly = TRUE)  
  on.exit(par(old_par))  
  
  par(mfrow = c(1, 2), mar = c(4, 4, 3, 1))  
  
  plot(  
    x,  
    type = "l",  
    main = paste0("AR(", length(phi), ")", n = "n", n),  
    xlab = "Time",
```

```

    ylab = "Value"
  )

  acf(x, main = "ACF")
}

```

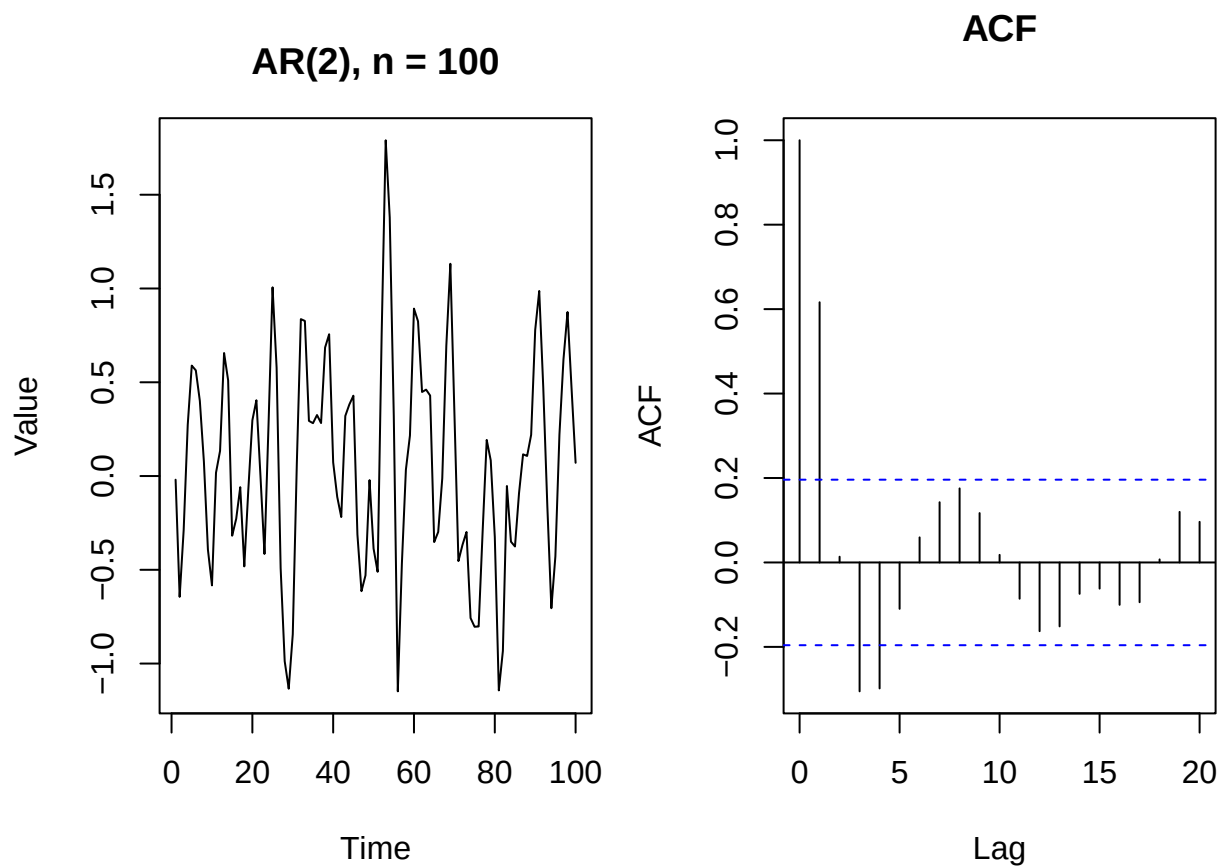
REPEAT FOR DIFFERENT n

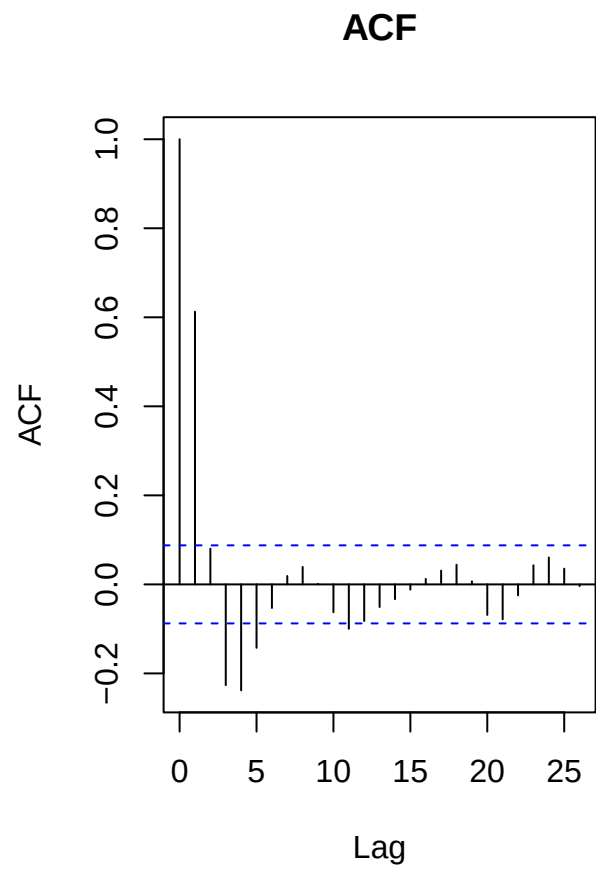
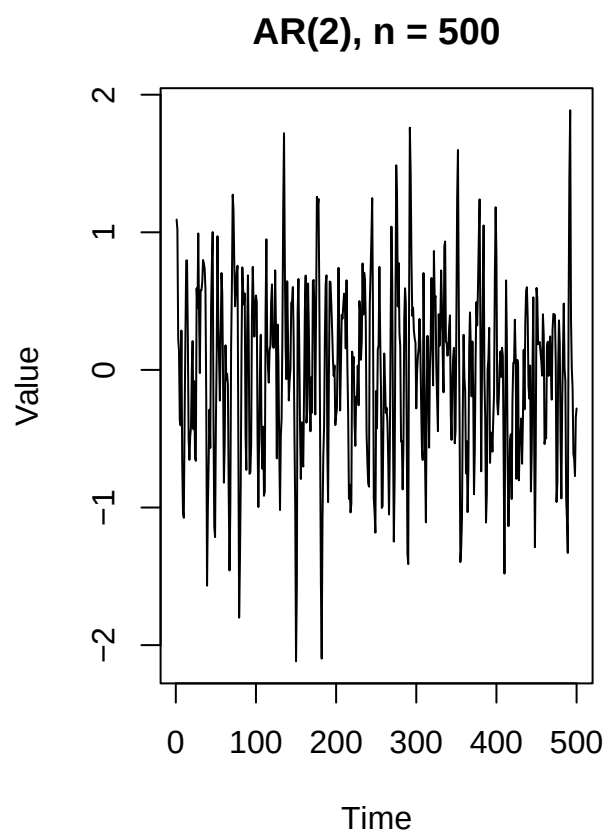
```

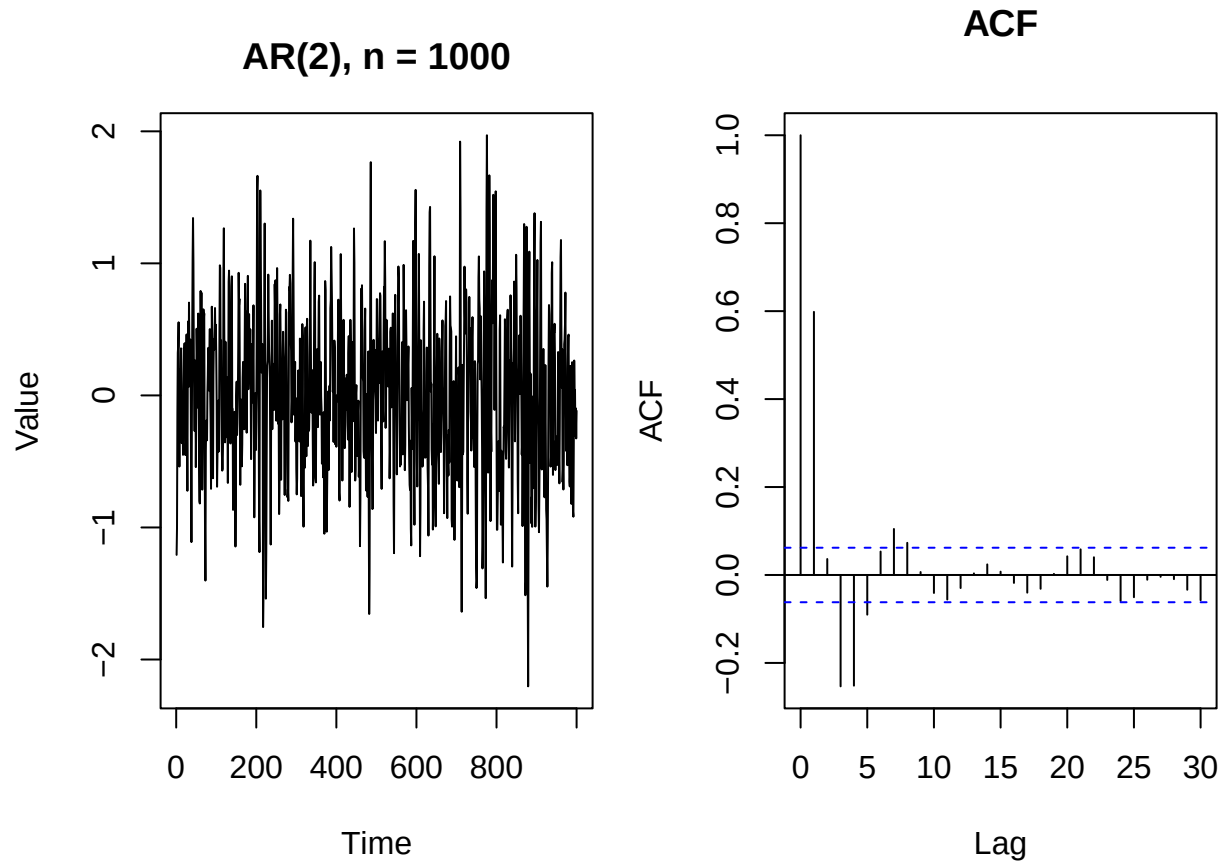
n_values <- c(100, 500, 1000)

for (n in n_values) {
  simulate_ar(n, phi = c(0.8897, -0.4858), sigma2 = 0.1796)
  dev.flush()
}

```







Observations:

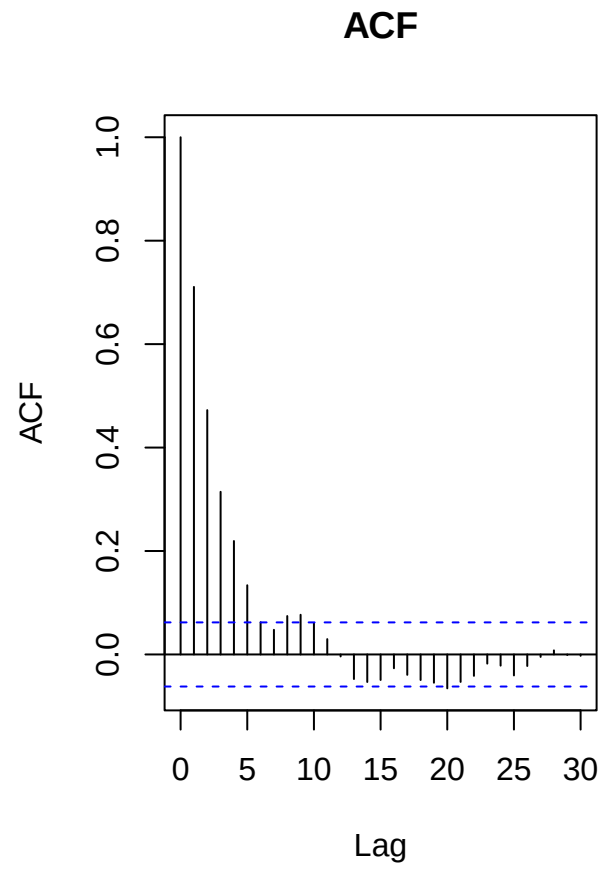
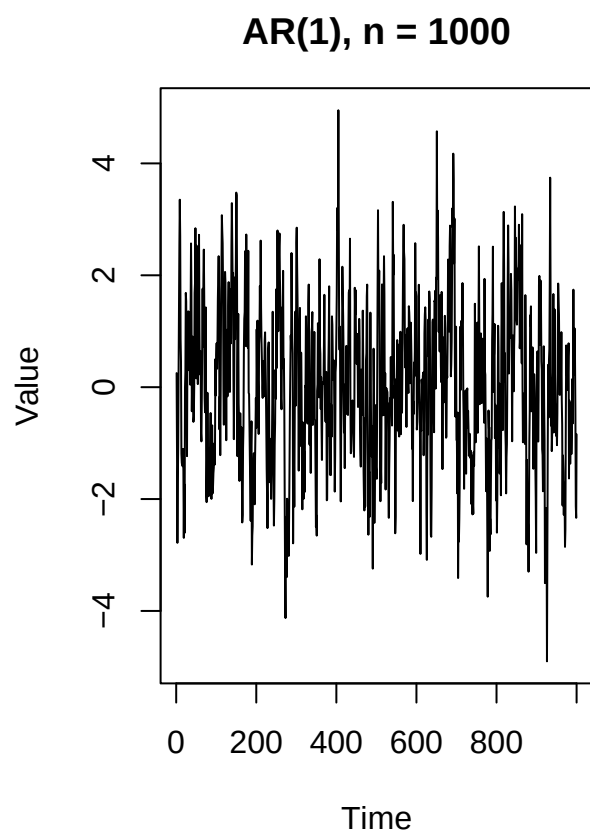
- Smaller n (100): the simulated series appears more irregular and the ACF is noisier. This makes the underlying auto-regressive structure less clear, because of the visual impact of the random fluctuations.
- Larger n (e.g. 1000): the behavior of the series becomes more representative of the theoretical AR model. The expected decay pattern of the ACF becomes clearer and smoother (after the decay, the values tend to be around 0, while for $n=100$ the values were still oscillating between -0.2 and 0.2 $\rightarrow$ dashed blue lines).

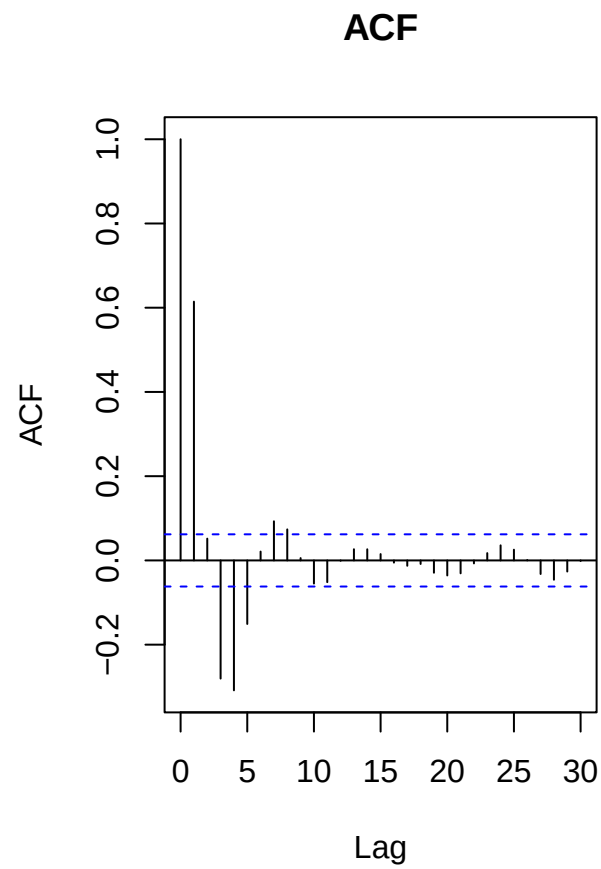
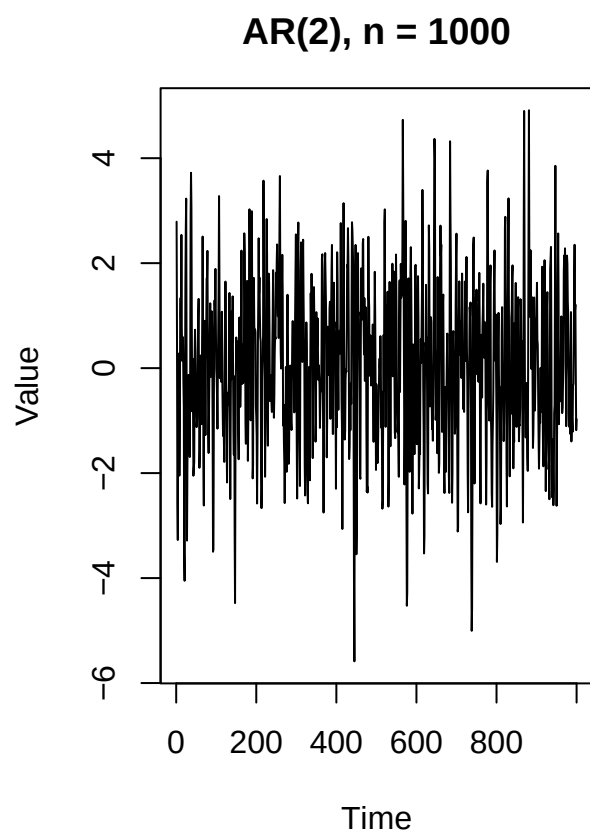
Overall, increasing n does not change the true model, but it improves the visibility of its properties.

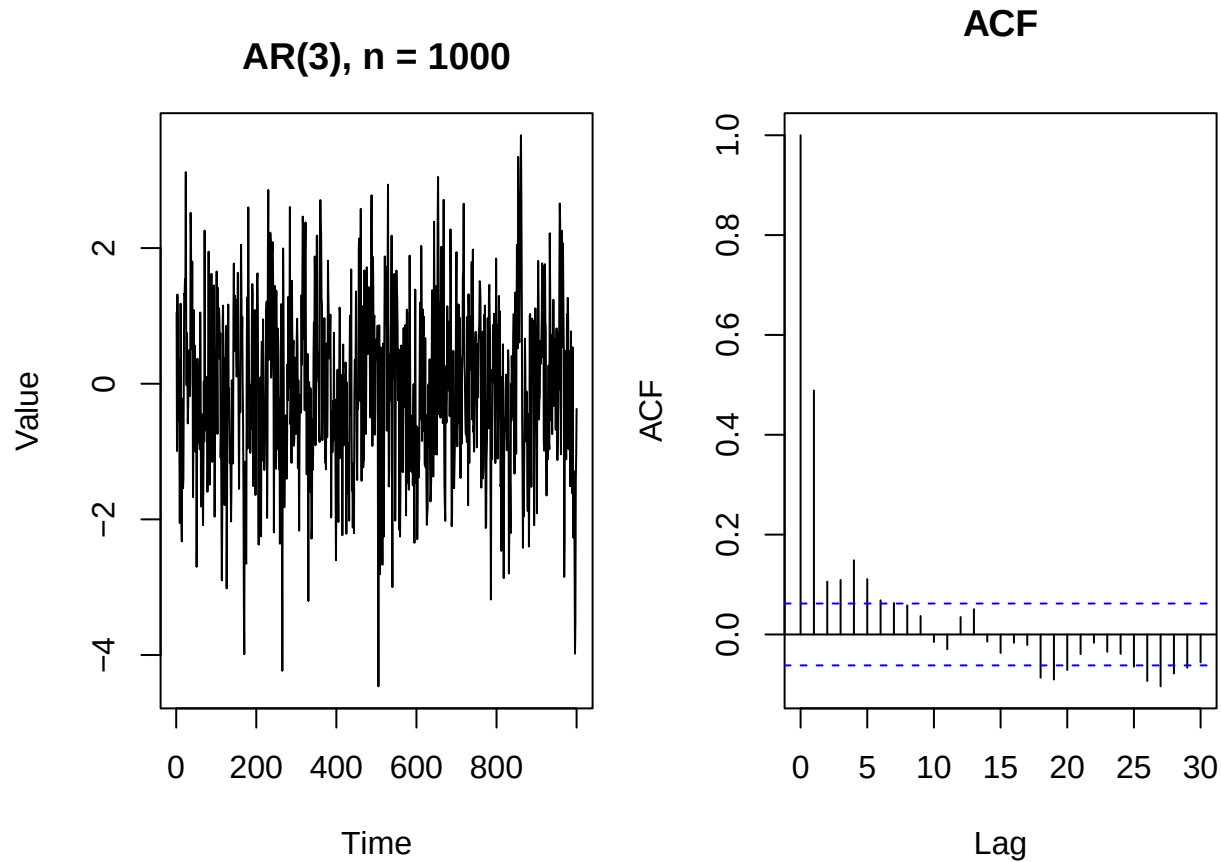
REPEAT FOR DIFFERENT AR orders

```
phi_list <- list(
  c(0.7), #AR(1)
  c(0.8897, -0.4858), #AR(2)
  c(0.6, -0.3, 0.2) #AR(3)
)

for (phi in phi_list) {
  simulate_ar(1000, phi)
}
```







Changing the order p changes how many past observations influence the current value. As p increases, the dependence structure becomes more complex and may persist over more lags.

- **AR(1):** each observation depends only on the immediately previous value. The ACF has a smooth decay.
- **AR(2) and AR(3):** the dependence involves more than one lag, so the series show richer dynamics. The ACF still decays gradually, but less smoothly than the AR(1) case and with an oscillatory behavior (resembling damped oscillations).

Overall, increasing the order of the model p increases the complexity of temporal dependence. This can be observed **both in the time series plot** (signals are richer as p increases) **and in the ACF**, which shows less smooth decay and more complex shapes due to the combined effect of multiple lagged terms.

REPEAT FOR DIFFERENT p

I choose to fix $n = 1000$ and $p=2$, so that I can observe how the time series and ACF change as I vary the parameters.

```
set.seed(123)
n <- 500

phi_list_values <- list(
  c(0.3, 0.2),
  c(0.8897, -0.4858),
```



```

c(1.2, -0.7)
)

titles_values <- c(
  "AR(2): phi = (0.3, 0.2)",
  "AR(2): phi = (0.8897, -0.4858)",
  "AR(2): phi = (1.2, -0.7)"
)

par(mfrow = c(3, 2), mar = c(4, 4, 3, 1))

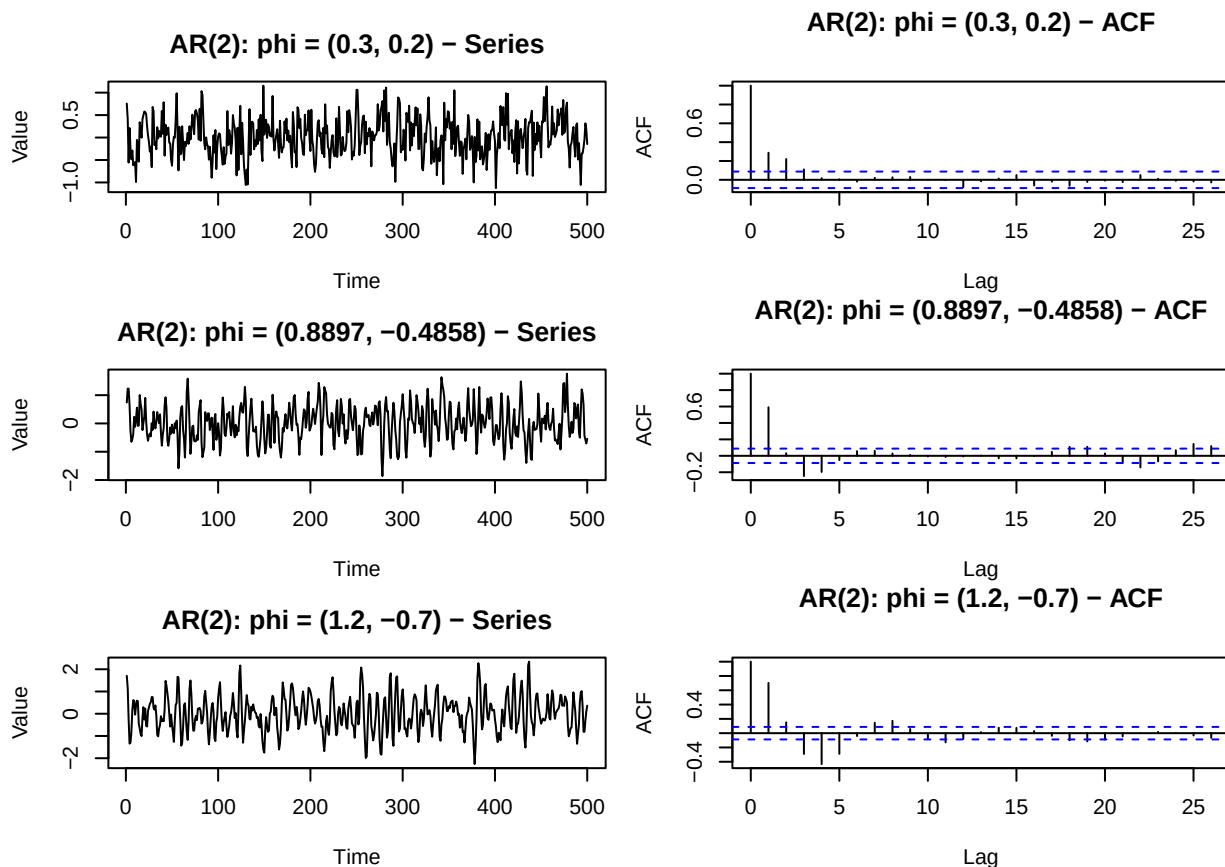
for (i in 1:length(phi_list_values)) {

  x <- arima.sim(list(ar = phi_list_values[[i]]), n = n, sd = sqrt(0.1796))

  plot(x, type = "l",
       main = paste(titles_values[i], "- Series"),
       xlab = "Time", ylab = "Value")

  acf(x,
      main = paste(titles_values[i], "- ACF"))
}

```



```

par(mfrow = c(1, 1))

```

Observations:

The three simulated AR(2) series show:

- similar overall behavior → in fact we can observe stationary fluctuations around a constant mean
- their dependence structure changes depending on the parameter values
 - $\phi = (0.3, 0.2)$ → the series looks “random” and the ACF goes to zero smoothly. This is the case with small coefficients, so the dependence between observations is weak.
 - $\phi = (0.8897, -0.4858)$ → the series looks slightly smoother than previous, the decay is a bit more oscillatory. The ACF stabilizes around 0 in longer time than $\phi = (0.3, 0.2)$ - which is in accordance with larger coefficients leading to stronger persistence.
 - $\phi = (1.2, -0.7)$ → the series looks more structured and less random, and the ACF has stronger oscillations. This could be indicative of alternating correlations across lags - which is compatible with such large coefficients.

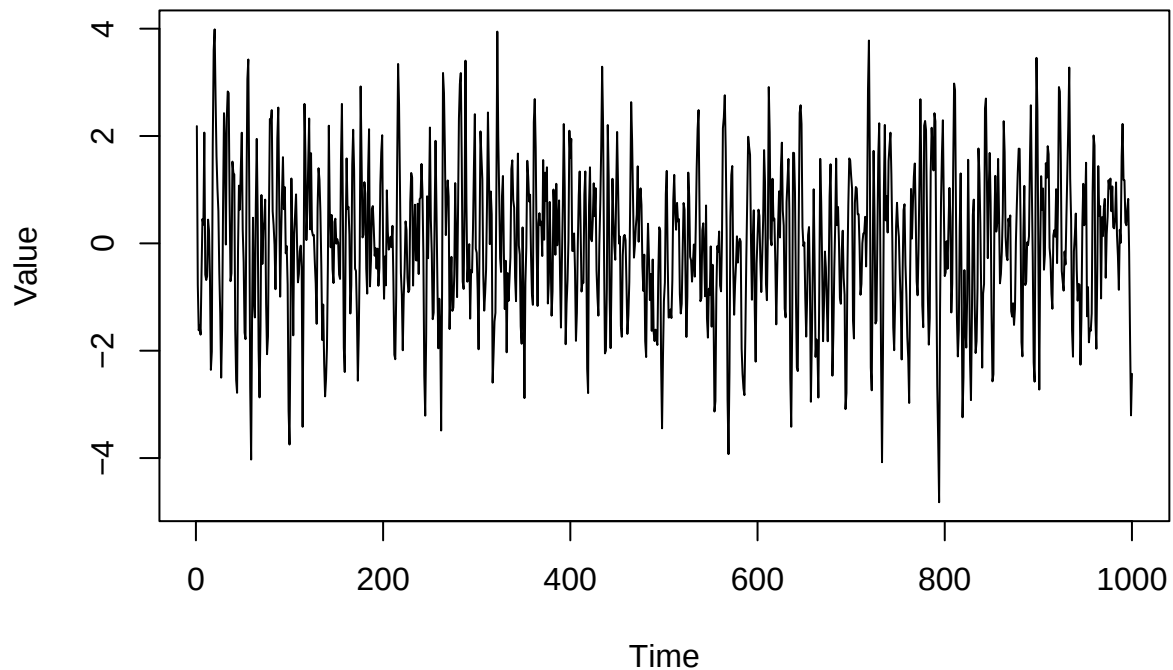
Overall, increasing the magnitude of the coefficients increases the persistence of the process. This also leads to oscillatory behavior in the ACF, which indicates a more complex temporal dependence.

2. Fitting AR(p) models

With the following commands, we will obtain an estimation of the AR(2) parameters and the intercept. We will assess the quality of the fit, and try with larger values of n .

```
n<-1000
x<-arima.sim(list(ar = c(0.8897, -0.4858), sd = sqrt(0.1796)), n)
plot(x,type="l",main="Simulated AR(2) series",xlab="Time",ylab="Value")
```

Simulated AR(2) series



```
s1<-arima(x, order=c(2,0,0), method="ML")
s1
```

```
##
## Call:
## arima(x = x, order = c(2, 0, 0), method = "ML")
##
## Coefficients:
##          ar1      ar2  intercept
##          0.8718 -0.5045      0.0011
## s.e.  0.0274   0.0275   0.0496
##
## sigma^2 estimated as 0.9851:  log likelihood = -1411.92,  aic = 2831.84
```

Observations:

- As it happened before, the generated time series fluctuates around a constant mean with no visible trend or seasonality, which is consistent with a stationary AR(2) process.
- **Fitted values:** these values estimate the autoregressive coefficients from the simulated data and they resulted to be very close to the true values
 - true coefficients: $1=0.8897$ and $2=-0.4858$
 - estimated coefficients: $1=0.8665$ and $2=-0.4481$So the model fits the data well.

Try with larger values of n

```
n_values <- c(100, 500, 1000, 5000)

par(mfrow = c(2,2), mar = c(4,4,3,1))

for (n in n_values) {
  x <- arima.sim(list(ar = c(0.8897, -0.4858), sd = sqrt(0.1796)), n = n)

  plot(x, type = "l",
       main = paste("Simulated AR(2), n =", n),
       xlab = "Time",
       ylab = "Value")

  s1 <- arima(x, order = c(2,0,0), method = "ML")

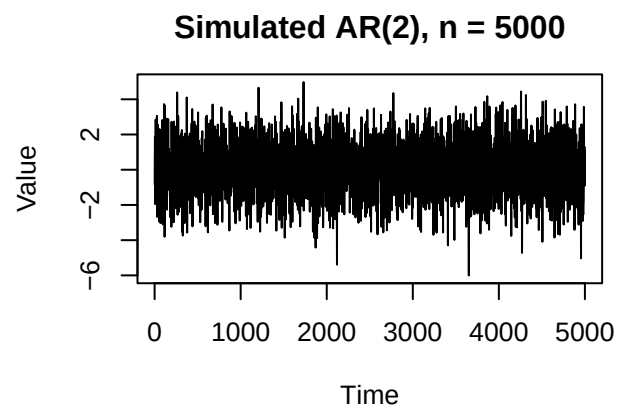
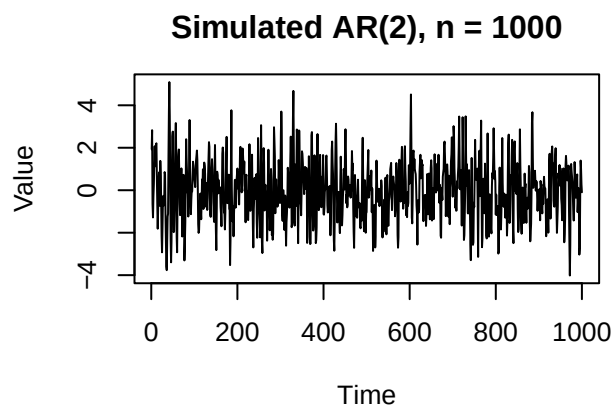
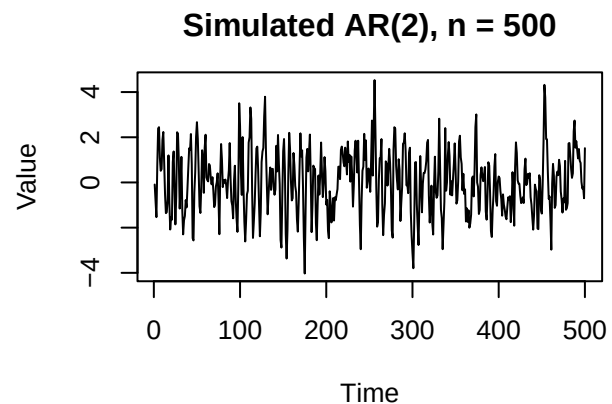
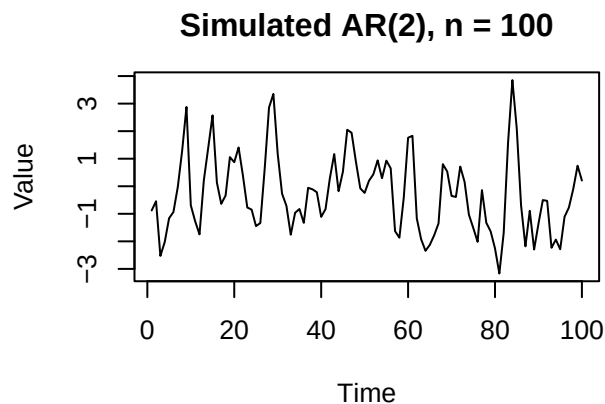
  cat("\n=====n")
  cat("n =", n, "\n")
  print(s1)
}
```

```
##
## =====
## n = 100
##
## Call:
## arima(x = x, order = c(2, 0, 0), method = "ML")
##
## Coefficients:
##          ar1      ar2  intercept
##          0.8891 -0.4485    -0.2877
## s.e.  0.0885   0.0881     0.1768
##
## sigma^2 estimated as 0.9775:  log likelihood = -141.22,  aic = 290.44

##
## =====
## n = 500
##
## Call:
## arima(x = x, order = c(2, 0, 0), method = "ML")
##
## Coefficients:
##          ar1      ar2  intercept
##          0.8421 -0.4239     0.0307
## s.e.  0.0406   0.0406     0.0776
##
## sigma^2 estimated as 1.02:  log likelihood = -714.85,  aic = 1437.69

##
## =====
## n = 1000
```

```
##
## Call:
## arima(x = x, order = c(2, 0, 0), method = "ML")
##
## Coefficients:
##          ar1          ar2  intercept
##      0.8582   -0.5019   -0.0196
## s.e.  0.0273    0.0274    0.0481
##
## sigma^2 estimated as 0.9593:  log likelihood = -1398.64,  aic = 2805.29
```



```
##
## =====
## n = 5000
##
## Call:
## arima(x = x, order = c(2, 0, 0), method = "ML")
##
## Coefficients:
##          ar1          ar2  intercept
##      0.8814   -0.4657    0.0029
## s.e.  0.0125    0.0125    0.0244
##
## sigma^2 estimated as 1.016:  log likelihood = -7135.94,  aic = 14279.88
```

```
par(mfrow = c(1,1))
```

Observations:

From the results above, we can notice that changing n brings to really different performance of the estimating model. As the number of samples n increases, the estimated AR coefficients gets closer to the real coefficients. In fact, the standard error (s.e.) for ar1, ar2 and intercept decreases. Moreover, as n increases, also the performance indicator *aic* increases. So we can conclude that if the model has more data points, it will be easier to be more accurate in estimating the autoregressive coefficients.

Do it several times simulating new series with different parameters

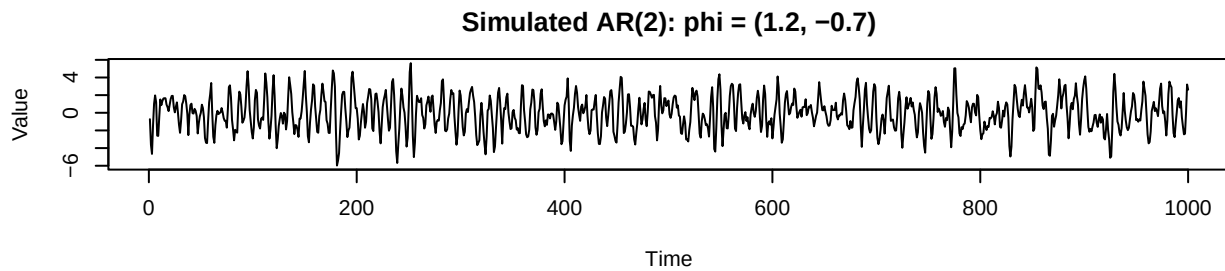
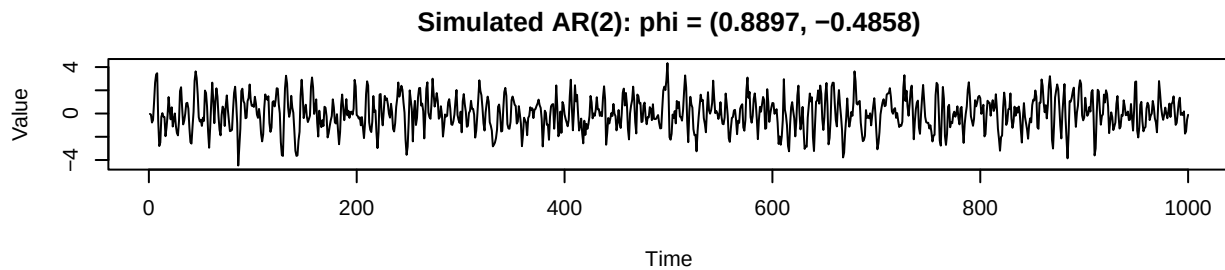
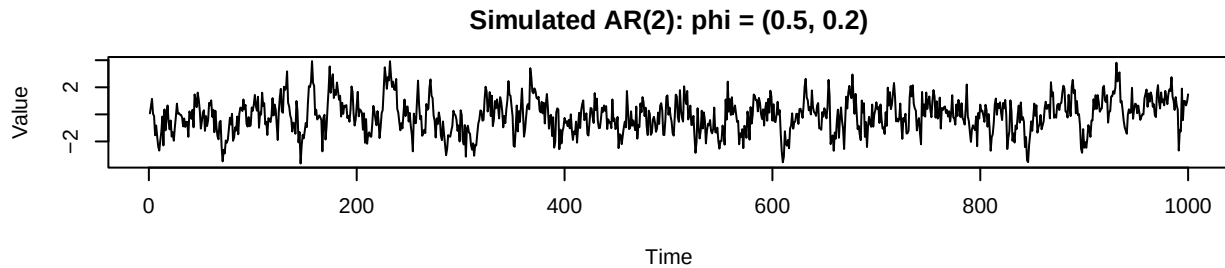
```
n <- 1000

phi_list <- list(
  c(0.5, 0.2),
  c(0.8897, -0.4858),
  c(1.2, -0.7)
)

par(mfrow = c(3,1), mar = c(4,4,3,1))

for (phi in phi_list) {
  x <- arima.sim(list(ar = phi, sd = sqrt(0.1796)), n = n)

  plot(x, type = "l",
       main = paste("Simulated AR(2): phi = (", paste(phi, collapse = ", "), ")", sep = ""),
       xlab = "Time",
       ylab = "Value")
}
```



```
par(mfrow = c(1,1))

for (phi in phi_list) {
  x <- arima.sim(list(ar = phi, sd = sqrt(0.1796)), n = n)
  s1 <- arima(x, order = c(2,0,0), method = "ML")

  cat("\n=====n")
  cat("True phi =", paste(phi, collapse = ", "), "\n")
  print(s1)
}
```

```
##
## =====
## True phi = 0.5, 0.2
##
## Call:
## arima(x = x, order = c(2, 0, 0), method = "ML")
##
## Coefficients:
##          ar1      ar2  intercept
##         0.4936  0.2424   -0.0499
## s.e.   0.0306  0.0307    0.1189
##
## sigma^2 estimated as 0.9925:  log likelihood = -1415.5,  aic = 2838.99
##
## =====
```

```
## True phi = 0.8897, -0.4858
##
## Call:
## arima(x = x, order = c(2, 0, 0), method = "ML")
##
## Coefficients:
##          ar1          ar2  intercept
##      0.8745   -0.4928     0.0207
## s.e.  0.0275    0.0275     0.0511
##
## sigma^2 estimated as 0.9985:  log likelihood = -1418.66,  aic = 2845.32
##
## =====
## True phi = 1.2, -0.7
##
## Call:
## arima(x = x, order = c(2, 0, 0), method = "ML")
##
## Coefficients:
##          ar1          ar2  intercept
##      1.2311   -0.7260     0.0375
## s.e.  0.0217    0.0217     0.0640
##
## sigma^2 estimated as 1.001:  log likelihood = -1420.68,  aic = 2849.37
```

Observations:

In this case as well, with different simulated time series, the fitting model performs well and estimates the autoregression coefficients which are very close to the true coefficients (all the standard errors on ar1, ar2, intercept are very small (around 0.01-0.06)).

This shows that the fitted model adapts to different structures: in all these cases it is able to recover the main autoregressive structure.

Try alternative method

Try instead `s2<- ar(x, order.max=2, method="mle")`.

```
n <- 1000
x <- arima.sim(list(ar = c(0.8897, -0.4858), sd = sqrt(0.1796)), n = n)

s1 <- arima(x, order = c(2,0,0), method = "ML")
s2 <- ar(x, order.max = 2, method = "mle")

s1
```

```
##
## Call:
## arima(x = x, order = c(2, 0, 0), method = "ML")
##
## Coefficients:
##          ar1          ar2  intercept
##      0.8819   -0.4792   -0.0780
## s.e.  0.0277    0.0278     0.0531
```



```
##
## sigma^2 estimated as 1.006:  log likelihood = -1422.62,  aic = 2853.25
```

```
s2
```

```
##
## Call:
## ar(x = x, order.max = 2, method = "mle")
##
## Coefficients:
##      1      2
## 0.8819 -0.4792
##
## Order selected 2  sigma^2 estimated as  1.006
```

Observations:

The two methods provide the same autoregressive coefficients and σ^2 .

The first method (s1) might be more helpful as it shows explicitly more details, e.g. the standard error and the intercept.

Try `order.max=3`

```
s3 <- ar(x, order.max = 3, method = "mle")
s3
```

```
##
## Call:
## ar(x = x, order.max = 3, method = "mle")
##
## Coefficients:
##      1      2
## 0.8819 -0.4792
##
## Order selected 2  sigma^2 estimated as  1.006
```

In this case the fitting model tries AR up to the 3rd order: AR(1), AR(2) and AR(3) and compares them using *aic* internally, to then choose the best one.

As the result shows, the best model chosen is still AR(2) and so it returned the same coefficients as before.

This makes sense because the time series that we are fitting, *x*, was generated from AR(2). So the model doesn't need to represent it with AR(3), indicating that adding an extra lag does not improve the model fit.

Overall, even when a higher maximum order is allowed, the model selection procedure correctly identifies AR(2) as the best model.

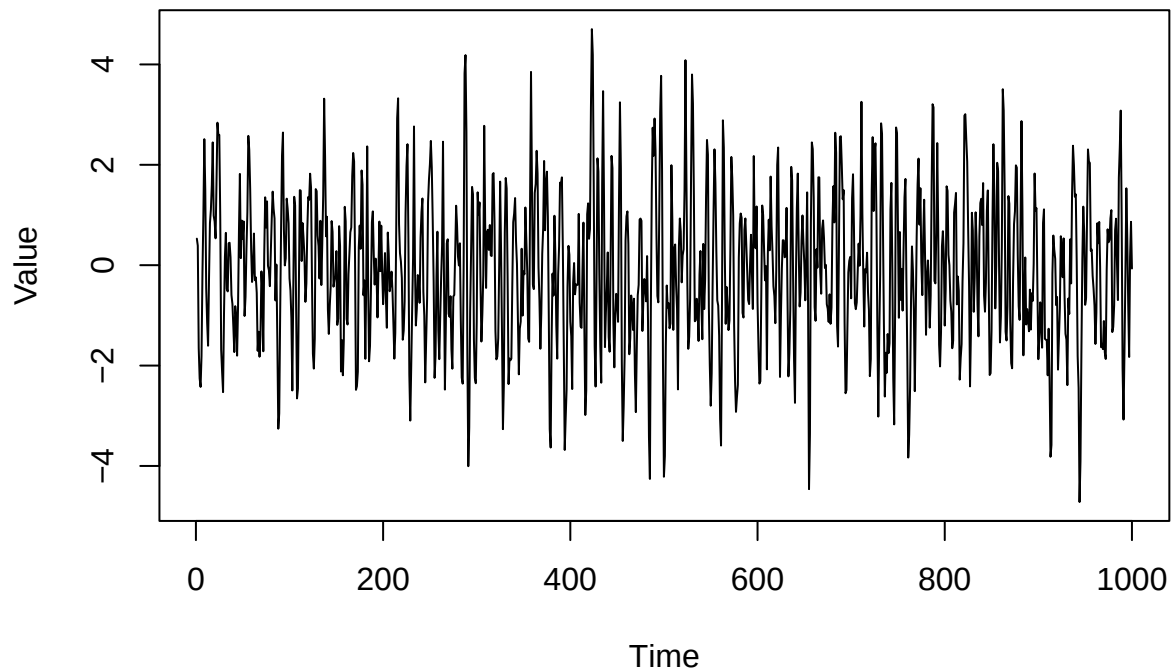
3. Validation

We will now experiment with the `Box.test` functionality.

First we create an AR(2) model with $\phi = (0.8897, -0.4858)$.

```
n<-1000
x<-arima.sim(list(ar = c(0.8897, -0.4858), sd = sqrt(0.1796)), n)
plot(x,type="l",main="Simulated AR(2) series, phi = (0.8897, -0.4858)",xlab="Time",ylab="Value")
```

Simulated AR(2) series, phi = (0.8897, -0.4858)

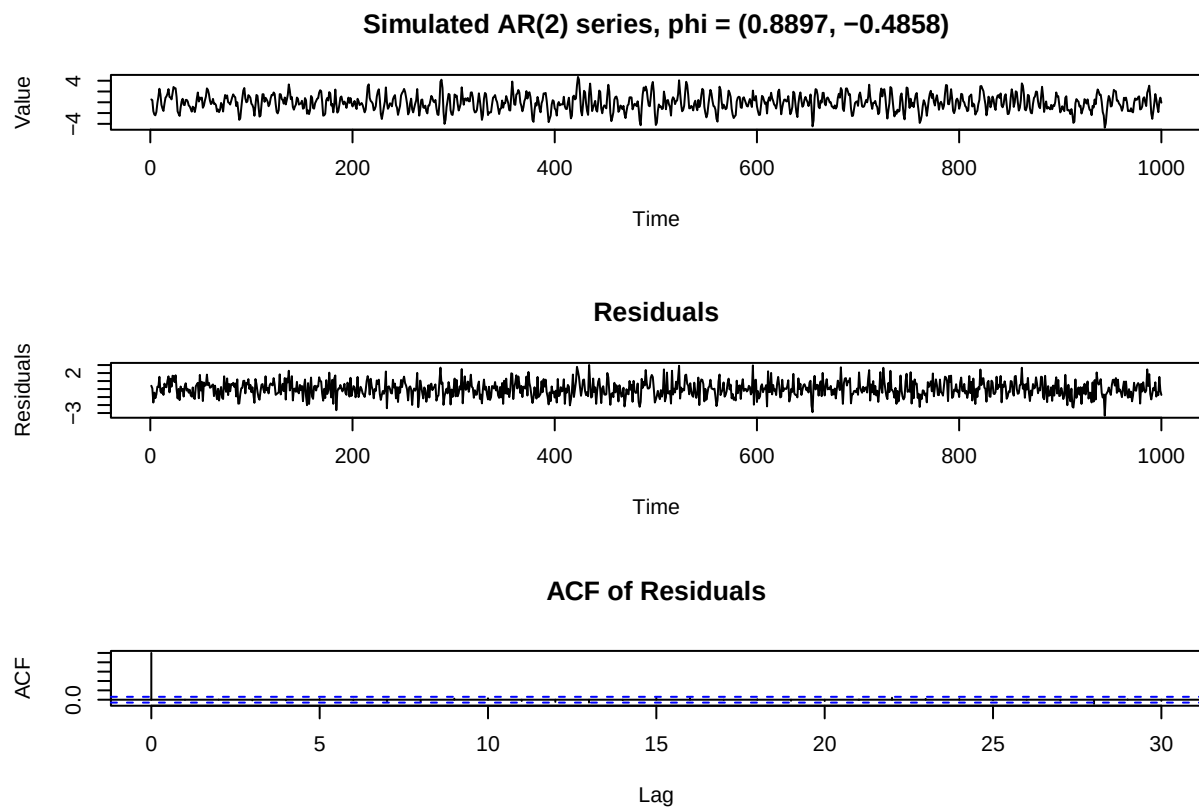


```
s<-arima(x, order=c(2,0,0), method="ML")
s
```

```
##
## Call:
## arima(x = x, order = c(2, 0, 0), method = "ML")
##
## Coefficients:
##          ar1      ar2  intercept
##       0.9369 -0.5303   -0.0809
## s.e.  0.0268  0.0268    0.0538
##
## sigma^2 estimated as 1.019:  log likelihood = -1429.1,  aic = 2866.2
```

```
y<-resid(s)
```

```
# Residuals plot + ACF together
par(mfrow = c(3,1))
plot(x,type="l",main="Simulated AR(2) series, phi = (0.8897, -0.4858)",xlab="Time",ylab="Value")
plot(y,type="l",main = "Residuals", xlab = "Time", ylab = "Residuals")
acf(y, main = "ACF of Residuals")
```



```
par(mfrow = c(1,1))

# Ljung-Box tests
h<-floor(log(n))
Box.test(x,lag=h, type=c("Ljung-Box"))

##
## Box-Ljung test
##
## data: x
## X-squared = 565.81, df = 6, p-value < 2.2e-16

Box.test(y,lag=h,type=c("Ljung-Box"))

##
## Box-Ljung test
##
## data: y
## X-squared = 0.69601, df = 6, p-value = 0.9946
```

Observations:

The residuals (y) fluctuate randomly around zero with no visible trend or structure, which suggests that the model has captured the main dynamics of the series.

Looking at the ACF, we don't notice significant spikes - indicating that there is no remaining autocorrelations. This suggests that the residuals behave like white noise.

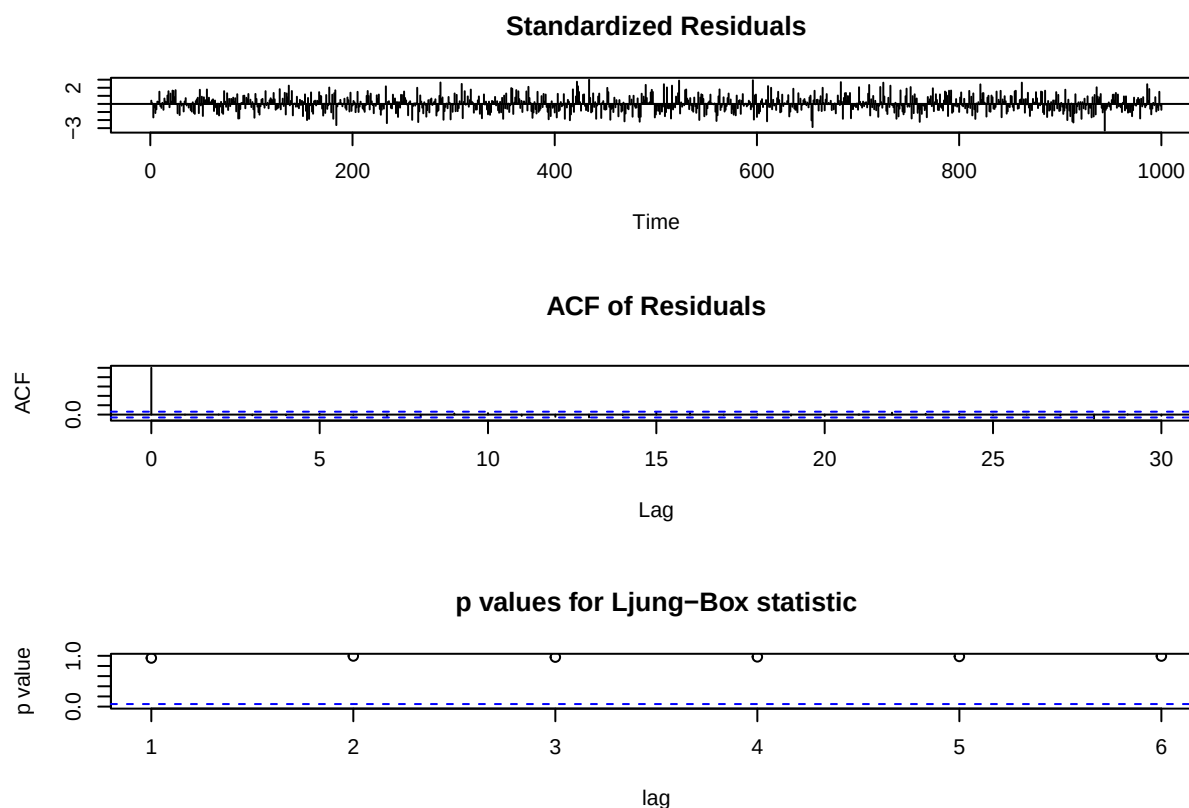
This is also confirmed by the result of the Box-Ljung test:

- **p-value of x** < 0.05 $\rightarrow$ the Ljung-Box test rejects the null hypothesis of independence, confirming that the series is not white noise and contains temporal dependence.
- **p-value of y** > 0.05 $\rightarrow$ the Ljung-Box test does not reject the null hypothesis, indicating that the residuals are consistent with white noise. This validates the fitted AR(2) model.

So eventually, since the residuals behave as white noise, we conclude that the AR(2) model provides an adequate fit to the data.

Alternative method:

```
h<-floor(log(n))
tsdiag(s,gof.lag=h)
```



The resulting plots confirm that the residuals behave as white noise. In particular, the p-values of the Ljung-Box test are generally above 0.05, indicating no significant autocorrelation remains.

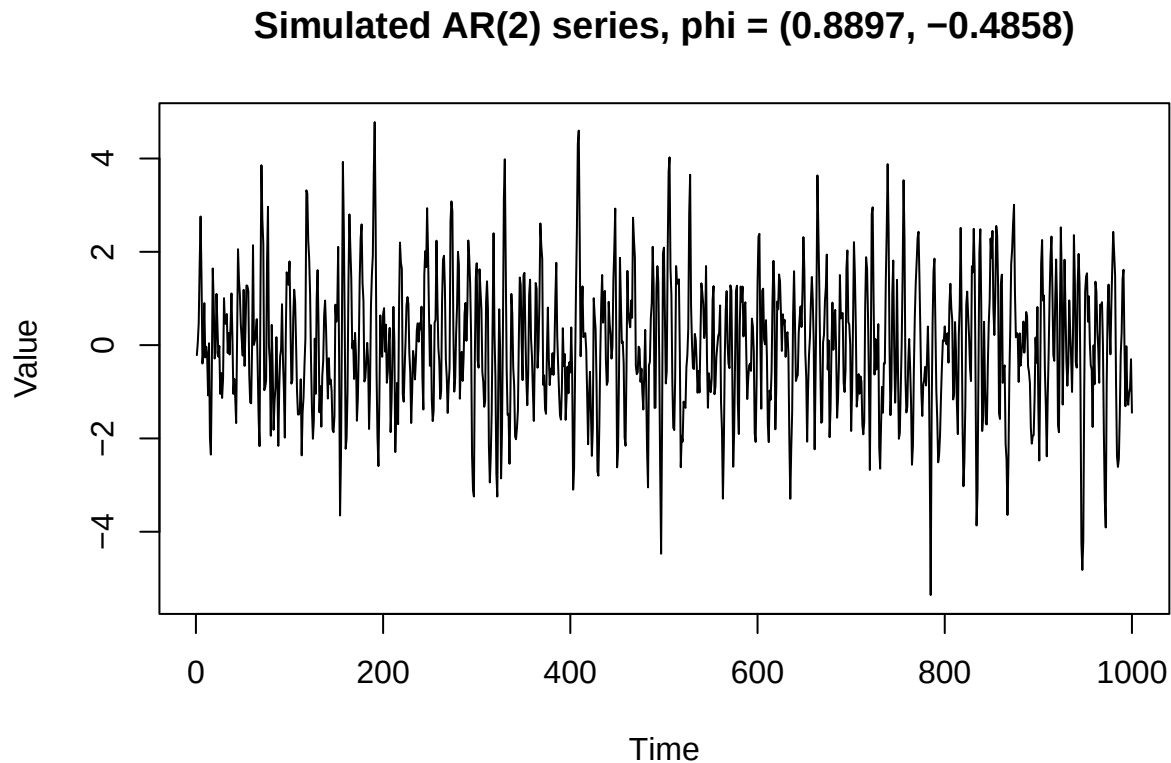
Conclusions:

The residual analysis shows that the fitted AR(2) model captures well the structure of the series. The residuals derived are randomly distributed, and their autocorrelation function shows no significant values. Furthermore, the Ljung-Box test rejects independence for the original series but not for the residuals. This confirms that the residuals behave as white noise and that the model provides a good fit.

4. Selection of the best AR(p) model

The following block selects the AR model, AR(1), AR(2) or AR(3), that best fits the data x , using the Akaike method, which we will see this in next lectures. The solution should be an AR(2).

```
n<-1000
x<-arima.sim(list(ar = c(0.8897, -0.4858), sd = sqrt(0.1796)), n)
plot(x,type="l",main="Simulated AR(2) series, phi = (0.8897, -0.4858)",xlab="Time",ylab="Value")
```



```
ss1<-ar(x, order.max=3, method="mle")
ss1

##
## Call:
## ar(x = x, order.max = 3, method = "mle")
##
## Coefficients:
##      1      2
## 0.9382 -0.4990
##
## Order selected 2  sigma^2 estimated as  0.946
```

As expected, even with the Akaike method the AR(2) is selected as best model to fit x .

5. A complete analysis with real data

We are going to use a dataset of the yearly level of Lake Huron from years 1875 to 1972.

The exercise suggests to create the timseries and plot it, along with its lag plots.

These are plots of $(x(i-1), x(i))$, then $(x(i-2), x(i))$, and so on. We are asked to notice that the linear trend disappears as we increase the lag.

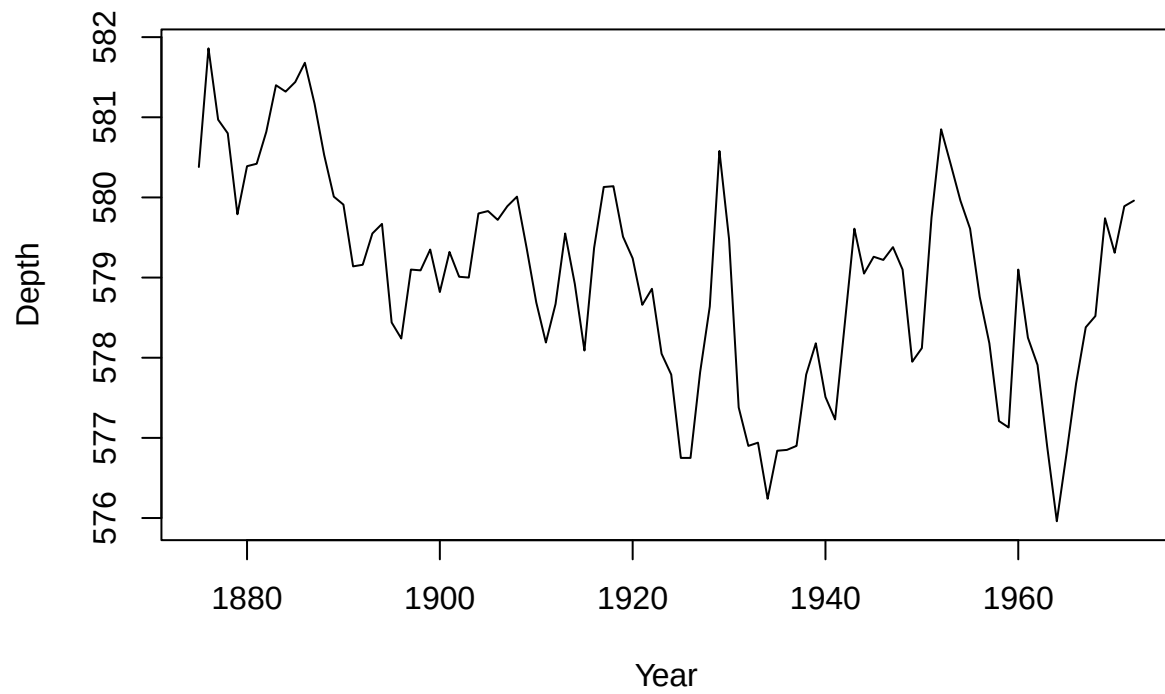
```
x<-LakeHuron
x
```

```
## Time Series:
## Start = 1875
## End = 1972
## Frequency = 1
## [1] 580.38 581.86 580.97 580.80 579.79 580.39 580.42 580.82 581.40 581.32
## [11] 581.44 581.68 581.17 580.53 580.01 579.91 579.14 579.16 579.55 579.67
## [21] 578.44 578.24 579.10 579.09 579.35 578.82 579.32 579.01 579.00 579.80
## [31] 579.83 579.72 579.89 580.01 579.37 578.69 578.19 578.67 579.55 578.92
## [41] 578.09 579.37 580.13 580.14 579.51 579.24 578.66 578.86 578.05 577.79
## [51] 576.75 576.75 577.82 578.64 580.58 579.48 577.38 576.90 576.94 576.24
## [61] 576.84 576.85 576.90 577.79 578.18 577.51 577.23 578.42 579.61 579.05
## [71] 579.26 579.22 579.38 579.10 577.95 578.12 579.75 580.85 580.41 579.96
## [81] 579.61 578.76 578.18 577.21 577.13 579.10 578.25 577.91 576.89 575.96
## [91] 576.80 577.68 578.38 578.52 579.74 579.31 579.89 579.96
```

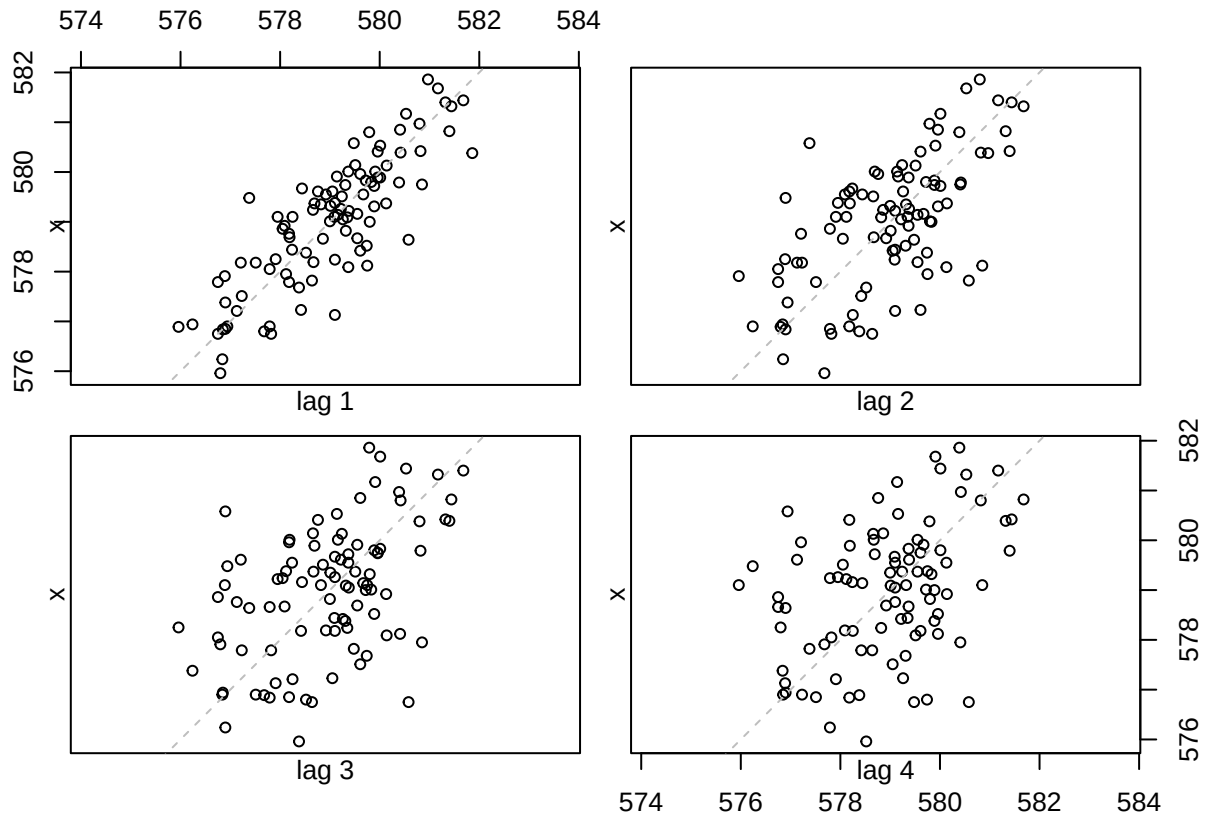
```
x <- LakeHuron

plot(x,main = "Lake Huron Water Levels (1875-1972)",xlab = "Year",ylab = "Depth")
```

Lake Huron Water Levels (1875–1972)



```
lag.plot (x, lag=4, do.lines=F)
```

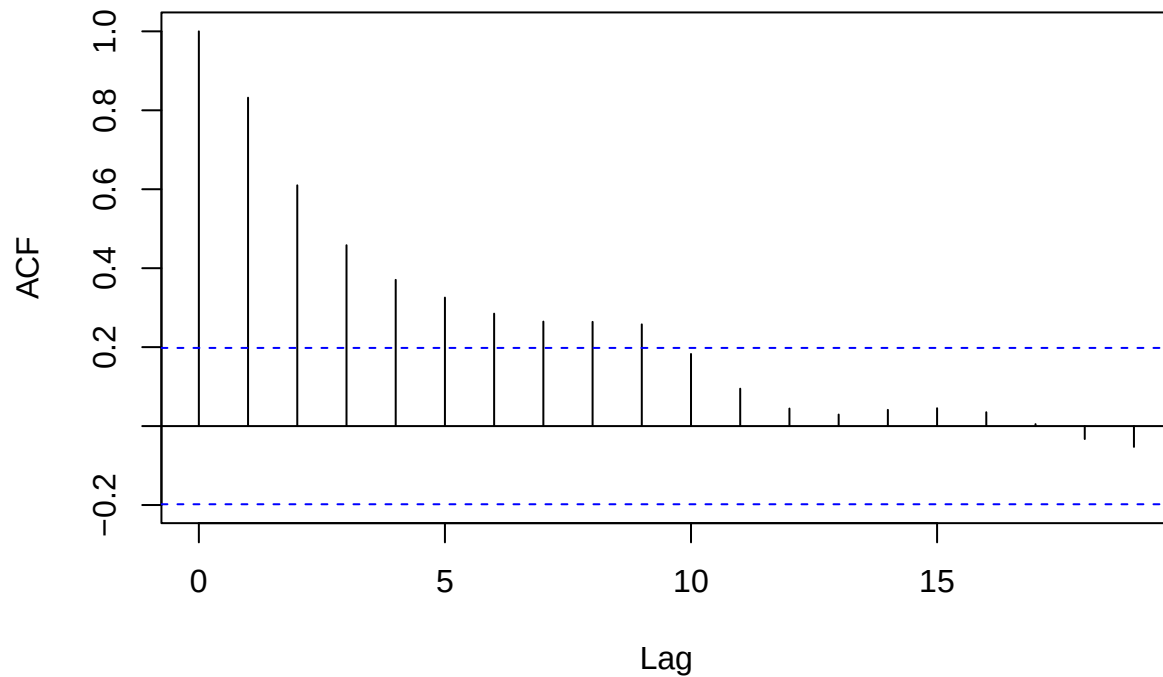


The time series represents yearly water levels of Lake Huron. The series appears relatively stable, although a slight downward trend can be observed over time, suggesting possible non-stationarity.

The **lag plots** show the relationship between the series and its past values. For small lags, there is a clear linear relationship, indicating strong dependence between consecutive observations. As the lag increases, the relationship becomes more dispersed, suggesting decreasing dependence.

`acf(x)`

Series x



The autocorrelation function shows a slow decay, indicating strong persistence in the series. This behavior suggests that the data may follow an autoregressive process and may not be purely random.

Now fit a model AR(1) to x , using Yule-Walker & Maximum Likelihood Estimation methods to estimate the parameters:

```
s1<-ar(x,order.max=1, method="yw")
s1
```

```
##
## Call:
## ar(x = x, order.max = 1, method = "yw")
##
## Coefficients:
##      1
## 0.8319
##
## Order selected 1  sigma^2 estimated as  0.5407
```

```
s2<-ar(x,order.max=1, method="mle")
s2
```

```
##
## Call:
## ar(x = x, order.max = 1, method = "mle")
##
```

```
## Coefficients:
##      1
## 0.8375
##
## Order selected 1  sigma^2 estimated as  0.5093
```

Fitting an AR(1) model using both Yule-Walker and Maximum Likelihood methods gives similar coefficient estimates. This indicates that both methods provide consistent results for this dataset.

Fit a generic AR(p) model

```
s3 <- ar(x, method = "mle")
s3
```

```
##
## Call:
## ar(x = x, method = "mle")
##
## Coefficients:
##      1      2
## 1.0437 -0.2496
##
## Order selected 2  sigma^2 estimated as  0.4788
```

```
s4 <- ar(x, method = "yw")
s4
```

```
##
## Call:
## ar(x = x, method = "yw")
##
## Coefficients:
##      1      2
## 1.0538 -0.2668
##
## Order selected 2  sigma^2 estimated as  0.5075
```

Also in this case we can notice that both MLE and YW give similar coefficient estimates. And they both select $p=2$, so they return AR(2) models. This indicates robustness in the model selection and estimation process.

Conclusions:

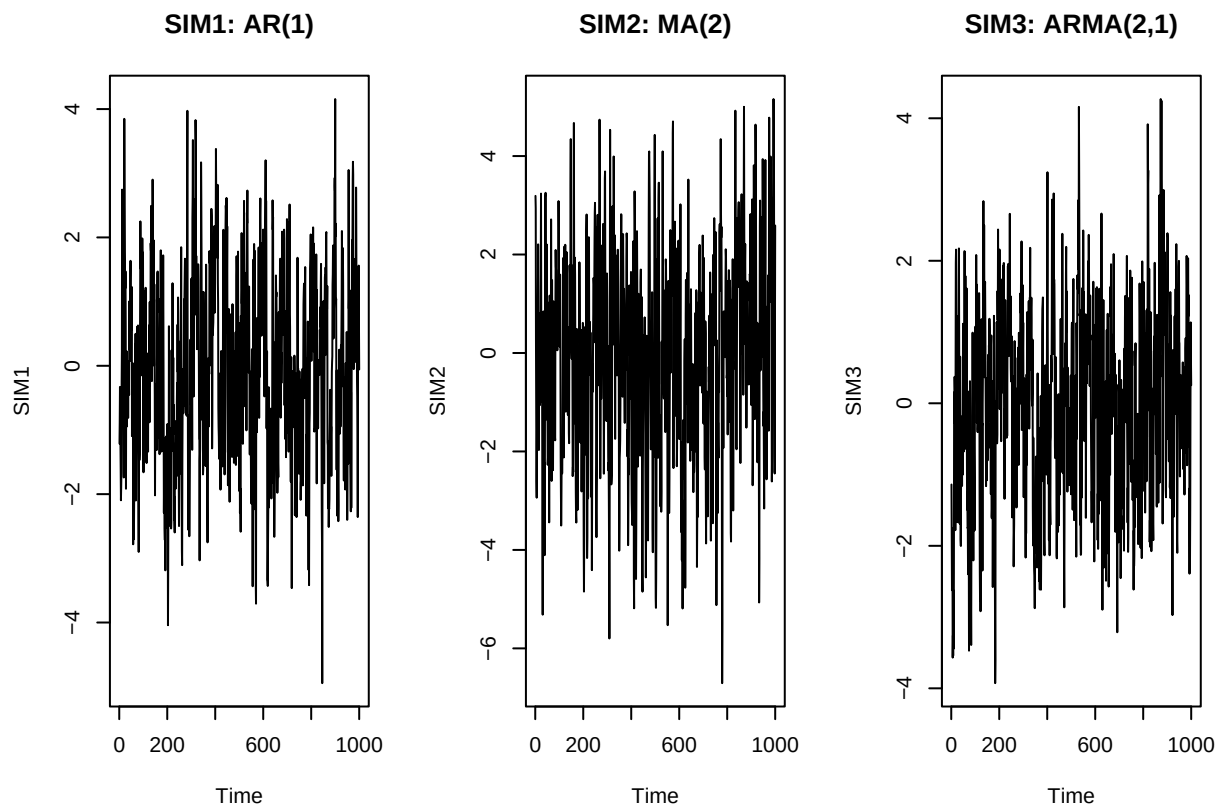
The analysis suggests that the Lake Huron time series exhibits autoregressive behavior, with an AR(2) model providing a better fit than AR(1). This is indeed coherent with what we saw in the ACF and lag plots: they both suggested that the data was not purely random but actually followed an autoregressive process.

Practical 3b Simulating and fitting ARMA models

1. Model simulation

```
# Simulate ARMA(1,0), ARMA(0,2) and ARMA(2,1) models
SIM1 <- arima.sim(list(order=c(1, 0, 0), ar=0.7), n=1000)
SIM2 <- arima.sim(list(order=c(0, 0, 2), ma=c(1.5, 0.75)), n=1000)
SIM3 <- arima.sim(list(order=c(2, 0, 1), ar=c(0.3, 0.2), ma=0.3), n=1000)

# Plot the simulated series
par(mfrow=c(1,3))
plot(SIM1, type="l", main="SIM1: AR(1)")
plot(SIM2, type="l", main="SIM2: MA(2)")
plot(SIM3, type="l", main="SIM3: ARMA(2,1)")
```



Interpretation:

- The AR(1) model (SIM1) shows an autocorrelation pattern, with values that tend to persist over time. Values depend on their immediate past, leading to smoother persistence.
- The MA(2) model (SIM2) exhibits more “spiky” behavior, with short memory.
- The ARMA(2,1) model (SIM3) combines features of both AR and MA, showing some persistence but also more variability than the pure AR(1) model.

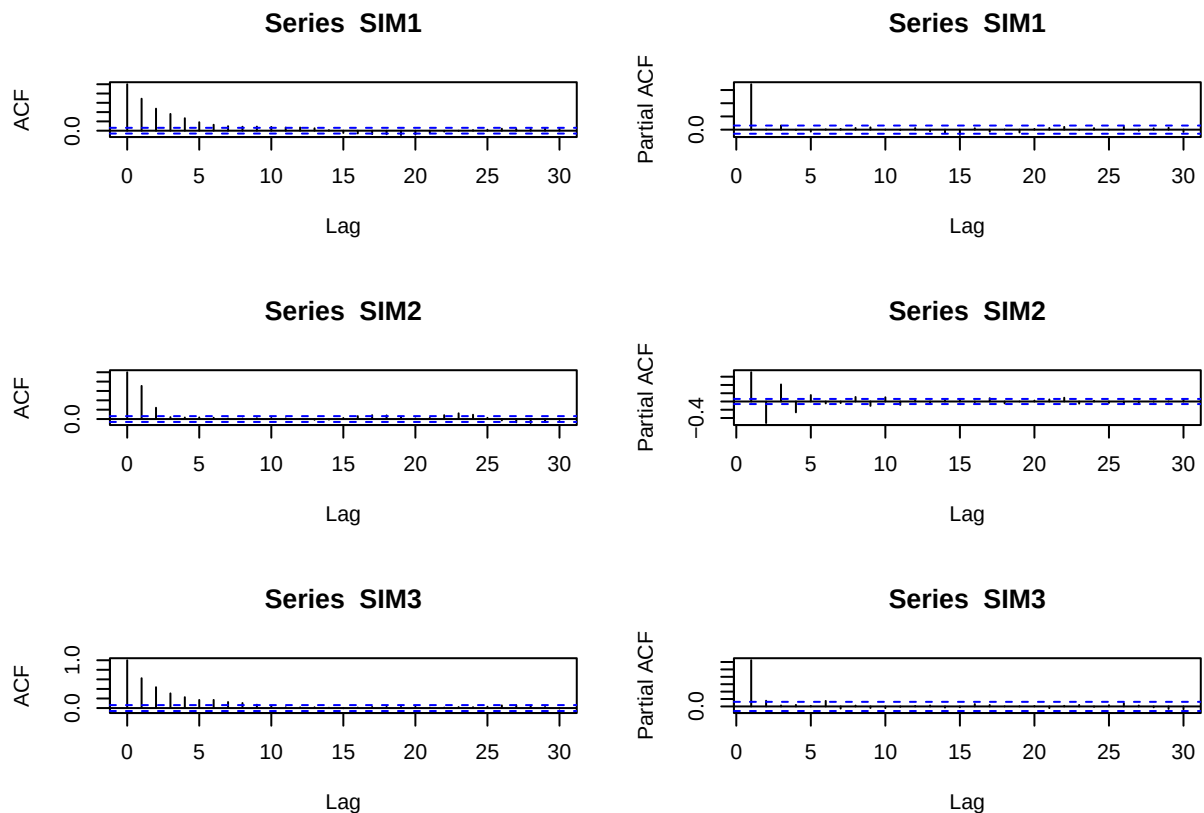
2. Heuristic identification of the model. Auto-correlation and partial auto-correlation functions

```
# Compute and plot the ACF and PACF for each simulated series
par(mfrow=c(3,2))

acf(SIM1, lag.max=30)
pacf(SIM1, lag.max=30)

acf(SIM2, lag.max=30)
pacf(SIM2, lag.max=30)

acf(SIM3, lag.max=30)
pacf(SIM3, lag.max=30)
```



Interpretation:

- For SIM1 (AR(1)), the ACF shows a gradual decay, while the PACF has a significant spike at lag 1 and then drops off.
- For SIM2 (MA(2)), the ACF has significant spikes at lags 1 and 2, while the PACF shows a gradual decay.
- For SIM3 (ARMA(2,1)), the ACF and PACF show a more complex pattern, with both decay and significant spikes, reflecting the combination of AR and MA components.

3. Fitting a model

```
# Fit ARMA models to the simulated data using maximum likelihood estimation
S1 <- arima(SIM1, order=c(1,0,0), method="ML")
S2 <- arima(SIM2, order=c(0,0,2), method="ML")
S3 <- arima(SIM3, order=c(2,0,1), method="ML")
```

```
# Display the fitted model summaries
```

S1

```
##
## Call:
## arima(x = SIM1, order = c(1, 0, 0), method = "ML")
##
## Coefficients:
##          ar1  intercept
##          0.6876   -0.0494
## s.e.   0.0229    0.1035
##
## sigma^2 estimated as 1.051:  log likelihood = -1443.9,  aic = 2893.8
```

S2

```
##
## Call:
## arima(x = SIM2, order = c(0, 0, 2), method = "ML")
##
## Coefficients:
##          ma1      ma2  intercept
##          1.5275  0.7618   -0.0739
## s.e.   0.0232  0.0217    0.1044
##
## sigma^2 estimated as 1.009:  log likelihood = -1425.07,  aic = 2858.15
```

S3

```
##
## Call:
## arima(x = SIM3, order = c(2, 0, 1), method = "ML")
##
## Coefficients:
##          ar1      ar2      ma1  intercept
##          1.1506  -0.2872  -0.5729   -0.058
## s.e.   0.3332   0.2203   0.3227    0.100
##
## sigma^2 estimated as 1.029:  log likelihood = -1433.4,  aic = 2876.81
```

Interpretation:

- The fitted AR(1) model for SIM1 show an estimated AR coefficient close to 0. (0.7172), confirming the true model.

- The fitted MA(2) model for SIM2 show estimated MA coefficients close to 1.5 (1.5178) and 0.75 (0.7475), confirming the true model.
- The fitted ARMA(2,1) model for SIM3 does not show estimated coefficients close to the true values (0.3, 0.2 for AR and 0.3 for MA). Obtained estimates are $\text{AR}(1) = 1.2707$, $\text{AR}(2) = -0.4478$ and $\text{MA}(1) = -0.6156$.

4. Validation

```
n <- length(SIM3)
h <- log(n)

Box.test(SIM3, lag=h, type="Ljung-Box")  # raw series
```

```
##
## Box-Ljung test
##
## data: SIM3
## X-squared = 785.87, df = 6.9078, p-value < 2.2e-16
```

```
y <- resid(S3)
Box.test(y, lag=h, type="Ljung-Box")  # residuals
```

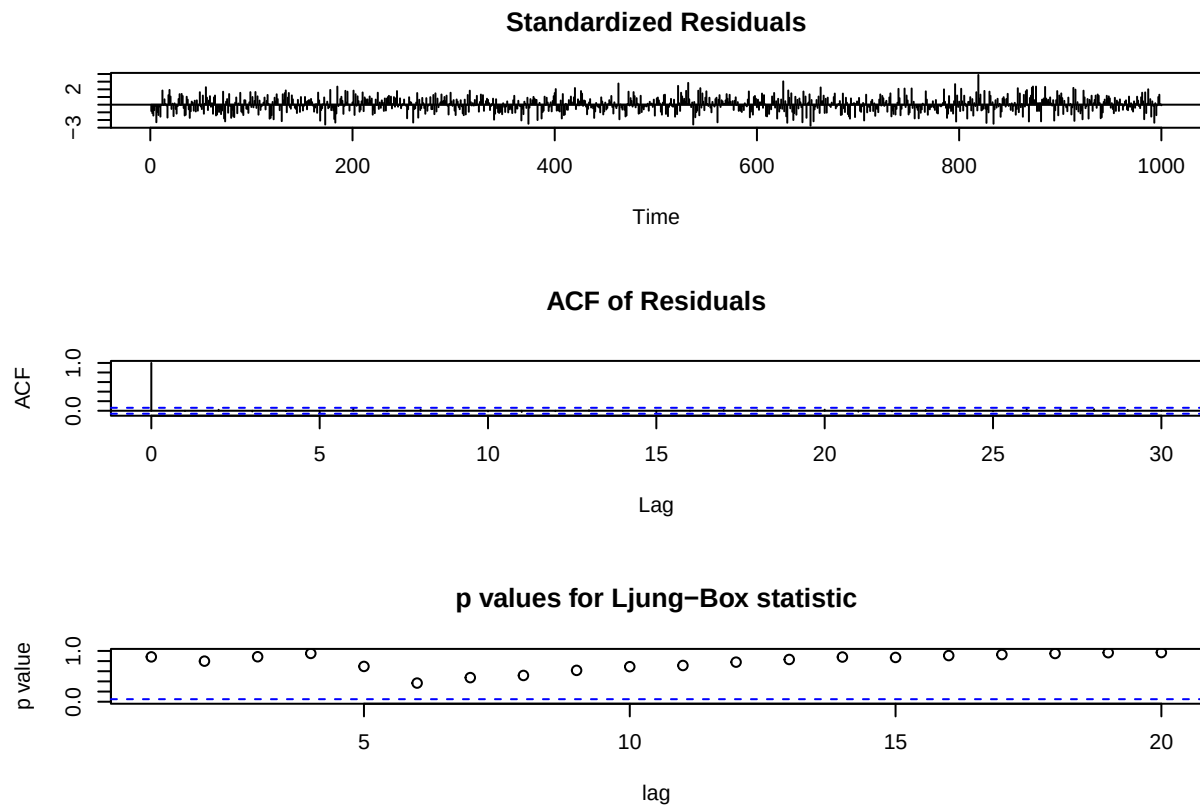
```
##
## Box-Ljung test
##
## data: y
## X-squared = 6.5236, df = 6.9078, p-value = 0.4697
```

Interpretation:

- The Ljung-Box test on the raw series (SIM3) yields a p-value of $2.2e-16$, rejecting the null hypothesis of independence, which is expected since SIM3 is an ARMA(2,1) model with autocorrelation and is not white noise.
- The Ljung-Box test on the residuals of the fitted model (S3) yields a p-value of 0.3582 which is greater than the common significance level of 0.05, failing to reject the null hypothesis of independence, indicating that the residuals behave like white noise and suggesting that the fitted ARMA(2,1) model adequately captures the autocorrelation structure of the data.

4.1. Faster method

```
# Diagnostic plots for the fitted ARMA(2,1) model
tsdiag(S3, gof.lag=20)
```



Interpretation:

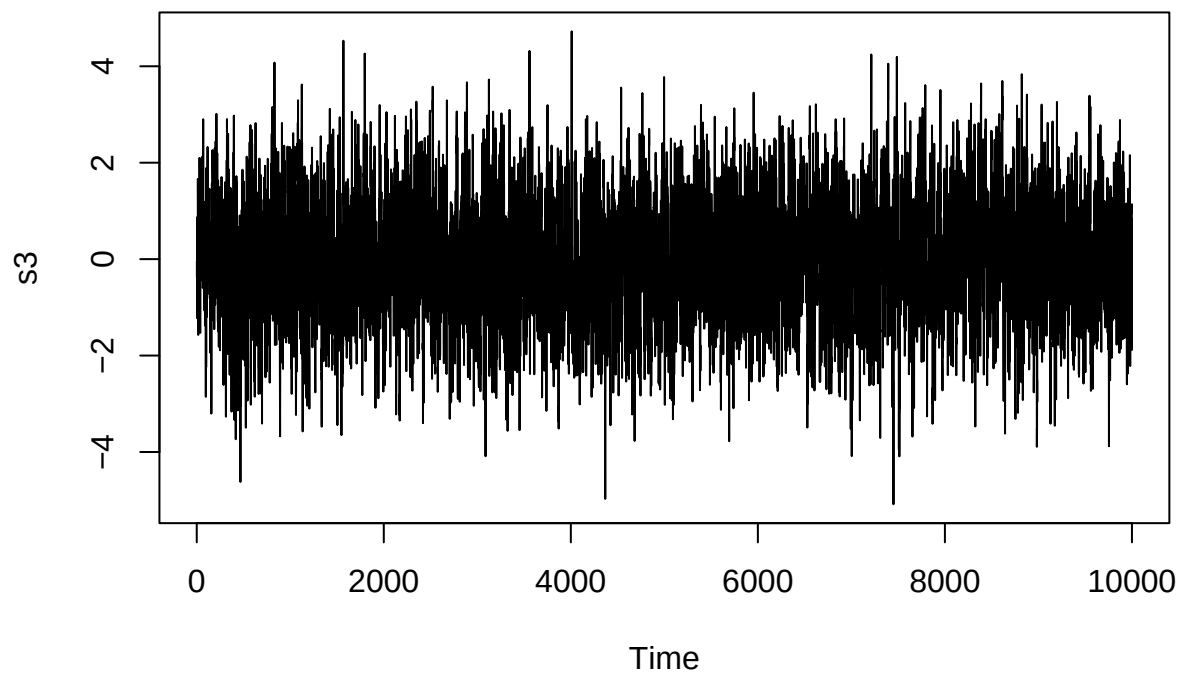
- The `tsdiag` function provides diagnostic plots for the fitted ARMA model (S3).
- The first plot shows the standardized residuals, which should ideally resemble white noise.
- The second plot displays the ACF of the residuals, where we look for no significant autocorrelations.
- The third plot shows the p-values of the Ljung-Box test for different lags, where we want to see p-values above 0.05, indicating that the residuals are not significantly autocorrelated.
- Overall, if the residuals appear random and the ACF shows no significant spikes, it suggests that the fitted ARMA(2,1) model is appropriate for the data.

5. A complete example

```
# Simulate an ARMA(2,1) model with n=10000 observations
n <- 10000
h <- log(n)

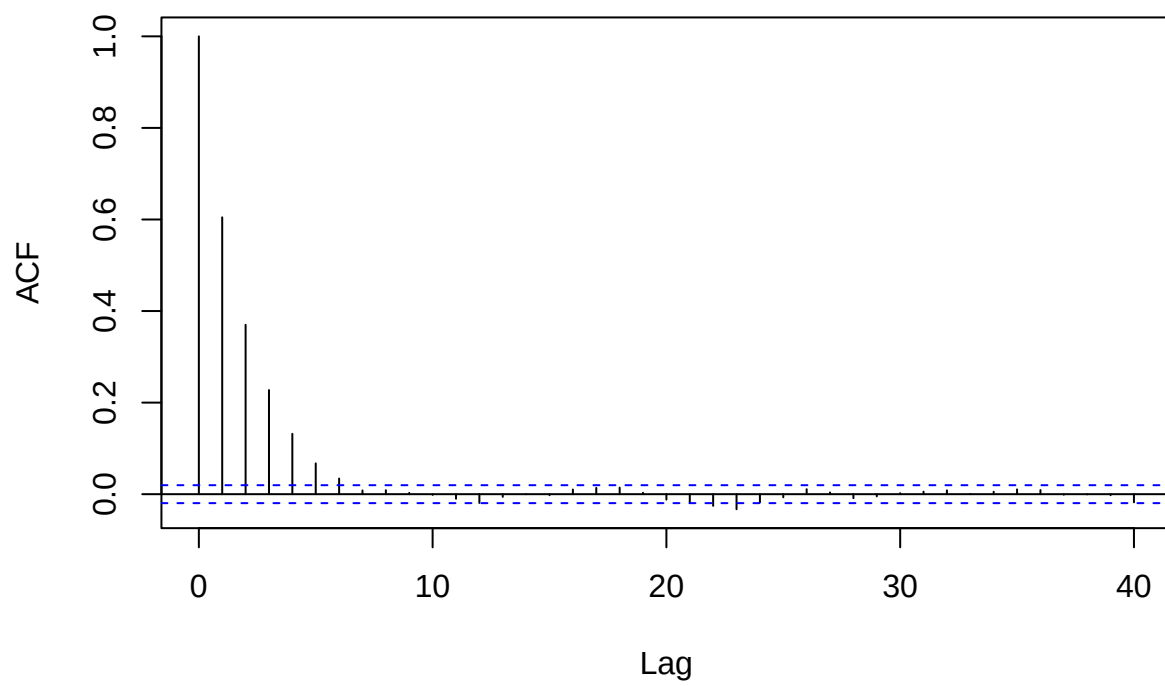
# Simulate the ARMA(2,1) model
s3 <- arima.sim(list(order=c(2,0,1), ar=c(0.3,0.2), ma=0.3), n)

# Plot the simulated series and its ACF and PACF
plot(s3, type="l")
```



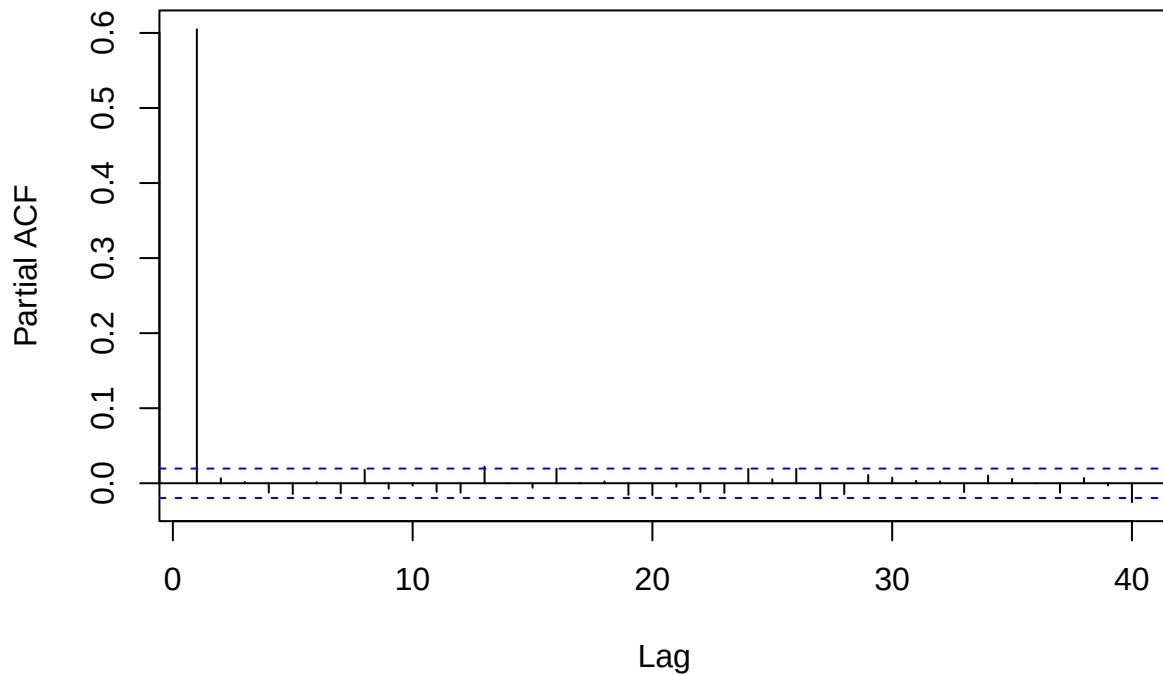
```
acf(s3)
```


Series s3



```
pacf(s3)
```

Series s3



```
# Fit the ARMA(2,1) model to the simulated data
p3 <- arima(s3, order=c(2,0,1), method="ML")
```

```
## Warning in arima(s3, order = c(2, 0, 1), method = "ML"): possible convergence
## problem: optim gave code = 1
```

```
p3
```

```
##
## Call:
## arima(x = s3, order = c(2, 0, 1), method = "ML")
##
## Coefficients:
##      ar1      ar2      ma1  intercept
##      0.4192  0.1156  0.1822   -0.0165
## s.e.  1.4644  0.8793  1.4705    0.0252
##
## sigma^2 estimated as 0.9874:  log likelihood = -14126.28,  aic = 28262.56
```

```
Box.test(s3, lag=h, type="Ljung-Box") # raw series
```

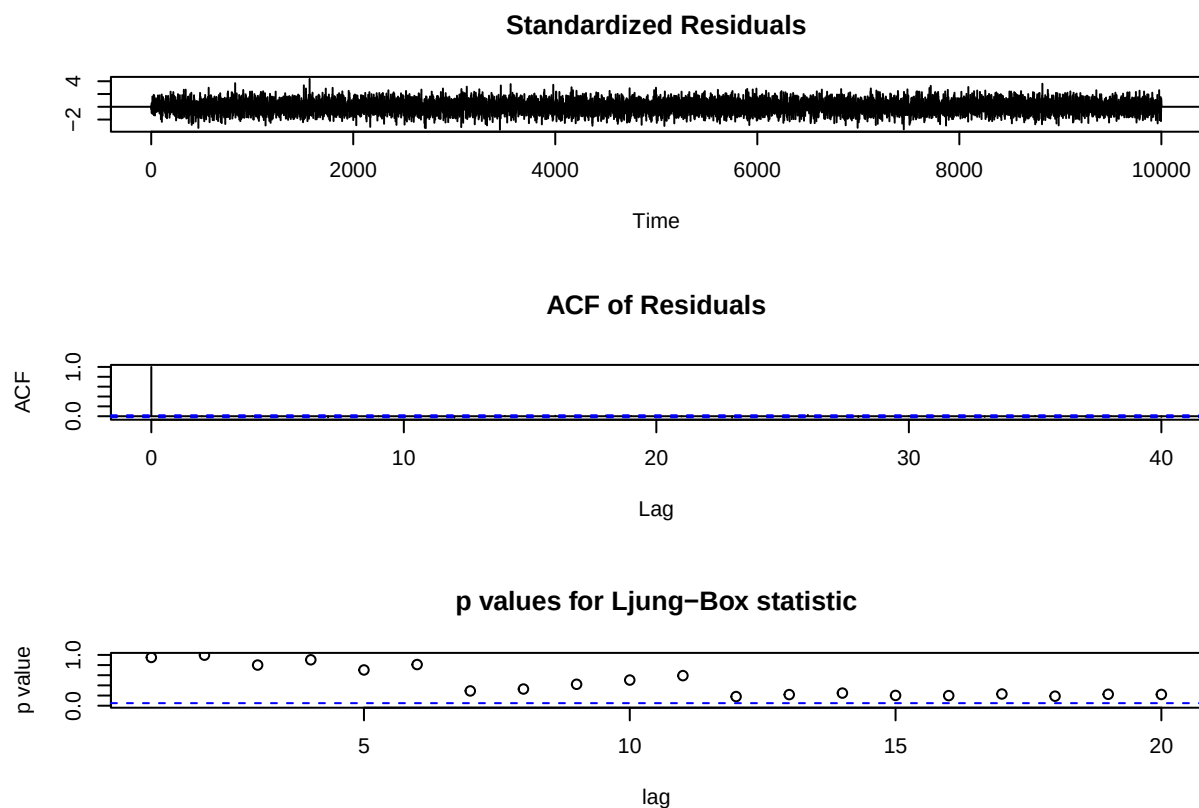
```
##
## Box-Ljung test
##
```

```
## data: s3
## X-squared = 5777.8, df = 9.2103, p-value < 2.2e-16
```

```
# Perform the Ljung-Box test on the residuals of the fitted model
Box.test(p3$residuals, lag=h, type="Ljung-Box")
```

```
##
## Box-Ljung test
##
## data: p3$residuals
## X-squared = 9.1722, df = 9.2103, p-value = 0.4415
```

```
tsdiag(p3, gof.lag=20)
```



Interpretation:

- As said before, the Ljung-Box test on the raw series (s3) yields a p-value of $2.2e-16$, rejecting the null hypothesis of independence
- The Ljung-Box test on the residuals of the fitted model (p3) yields a p-value of 0.8576, which is greater than the common significance level of 0.05, failing to reject the null hypothesis of independence, indicating that the residuals behave like white noise and suggesting that the fitted ARMA(2,1) model adequately captures the autocorrelation structure of the data.

6. Roots of Polynomials

Given a polynomial $1 + a_1 z + a_2 z^2 + \dots + a_p z^p$, we can obtain its roots using the `polyroot` function in R. For an AR(2) model with parameters (0.8897, -0.4858), we can compute the roots as follows:

```
# Coefficients of the polynomial
a <- c(1, -0.8897, 0.4858)
# Compute the roots
roots <- polyroot(a)
roots
```

```
## [1] 0.9157061+1.10451i 0.9157061-1.10451i
```

7. Download the daily closing prices for Siemens Energy and store them to a SE.csv file

```
# import libraries
import yfinance as yf
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import requests
from statsmodels.tsa.stattools import adfuller
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.stats.diagnostic import acorr_ljungbox
from statsmodels.tsa.statespace.sarimax import SARIMAX
from datetime import datetime
import warnings
warnings.filterwarnings("ignore")
```

7.1. Download the data

```
ticker = "ENR.DE" # Siemens Energy ticker on XETRA
data = yf.download(ticker, start="2020-01-01", end="2025-12-31")
```

```
## [*****100%*****] 1 of 1 completed
```

```
data.to_csv("SE.csv") # save to CSV
print(data.head())
```

## Price	Close	High	Low	Open	Volume
## Ticker	ENR.DE	ENR.DE	ENR.DE	ENR.DE	ENR.DE
## Date					
## 2020-09-29	21.794153	23.151335	21.358270	21.645557	10261596
## 2020-09-30	22.784796	22.953205	21.368176	21.784246	5914951
## 2020-10-01	22.398447	23.547593	22.398447	23.181054	4359302
## 2020-10-02	22.210224	22.755077	21.774341	22.190411	4092751
## 2020-10-05	21.546494	22.566857	21.546494	22.348914	3199904

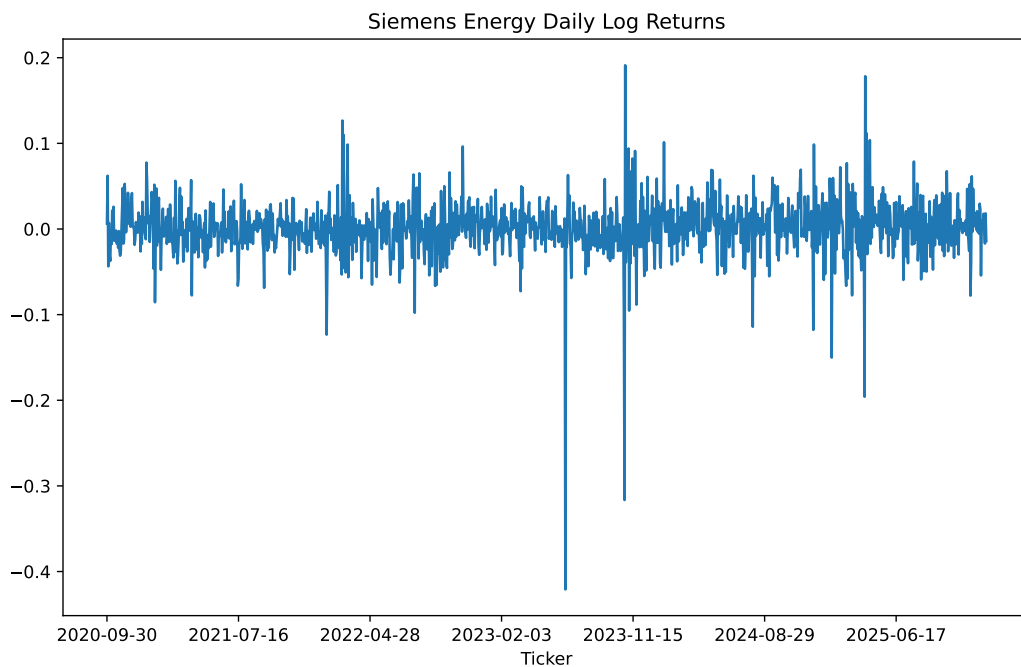
7.2. Load the CSV and compute log returns

```
# yfinance has extra row at the beginning, skip it with header=1
se_data = pd.read_csv("SE.csv", header=1, index_col=0, parse_dates=True)
se_data.columns = se_data.columns.str.strip()
print(se_data.columns)

## Index(['ENR.DE', 'ENR.DE.1', 'ENR.DE.2', 'ENR.DE.3', 'ENR.DE.4'], dtype='str')

# Compute daily log returns
# log_return[t] = log(Close_t) - log(Close_{t-1})
returns = np.log(se_data['ENR.DE.3']).diff().dropna()

# Plot
returns.plot(title="Siemens Energy Daily Log Returns", figsize=(10,6))
plt.show()
```



7.3. Check stationarity with ADF test

```
adf_result = adfuller(returns)
print(f"ADF Statistic: {adf_result[0]}")

## ADF Statistic: -40.30339477269587
```

```
print(f"p-value: {adf_result[1]}")
```

```
## p-value: 0.0
```

```
print("Critical Values:", adf_result[4])
```

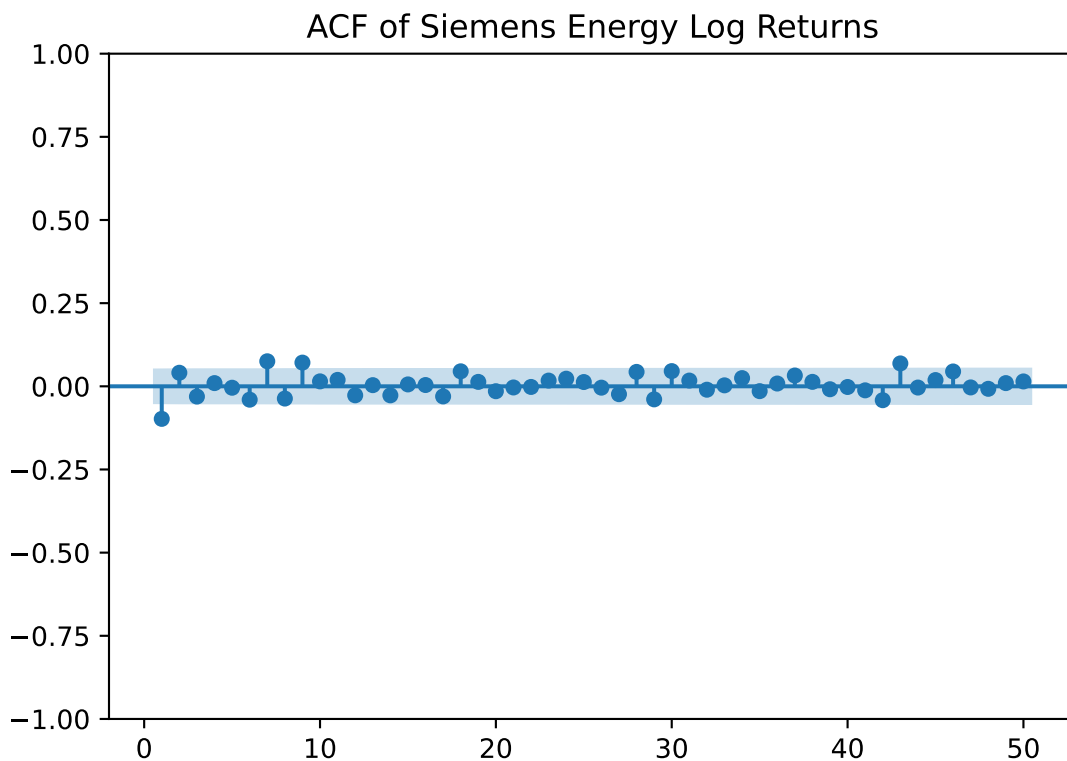
```
## Critical Values: {'1%': np.float64(-3.435250448324718), '5%': np.float64(-2.8637041654378903), '10%':
```

Interpretation:

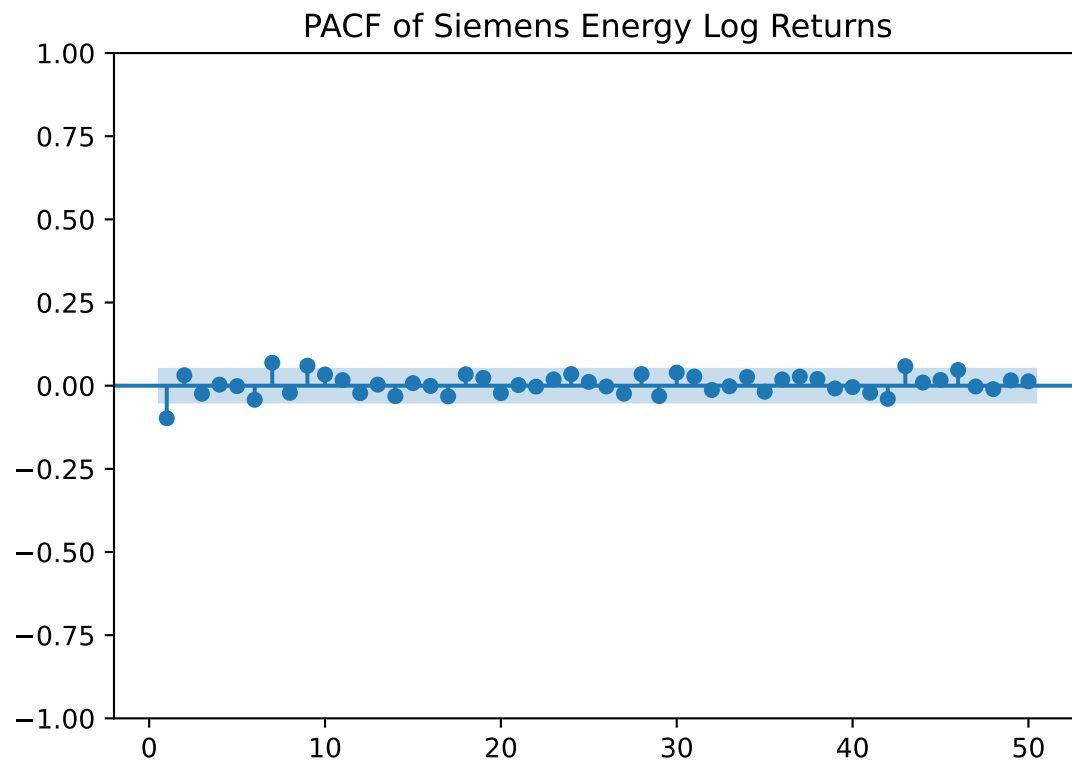
- The log returns are stationary because the ADF test statistic is less than the critical value and the p-value is less than 0.05
- ADF Statistic: -40.30339446788923
- p-value: 0.0
- We can reject the null hypothesis of a unit root, confirming that the log returns are stationary.

7.4. Plot ACF and PACF

```
# Plot ACF  
plt.figure(figsize=(12,5))  
plot_acf(returns, lags=50, zero=False)  
plt.title("ACF of Siemens Energy Log Returns")  
plt.show()
```



```
# Plot PACF
plt.figure(figsize=(12,5))
plot_pacf(returns, lags=50, zero=False, method='ywmm') # 'ywmm' handles large series better
plt.title("PACF of Siemens Energy Log Returns")
plt.show()
```



7.5. Fit an ARMA(1, 1) and ARMA(2, 1) model based on ACF and PACF

```
# ARMA(p, q) is ARIMA(p, 0, q) in statsmodels
model_11 = ARIMA(returns, order=(1, 0, 1)) # p=1, d=0, q=1
fit_11 = model_11.fit()
print(fit_11.summary())
```

```
##                                SARIMAX Results
## =====
## Dep. Variable:                ENR.DE.3    No. Observations:                1338
## Model:                      ARIMA(1, 0, 1)  Log Likelihood                2686.852
## Date:                      Fri, 27 Mar 2026  AIC                -5365.703
## Time:                      18:35:49      BIC                -5344.908
## Sample:                    0              HQIC                -5357.912
##                               - 1338
## Covariance Type:            opg
## =====
##              coef      std err          z      P>|z|      [0.025      0.975]
## -----
## const          0.0013      0.001        1.334      0.182      -0.001      0.003
## ar.L1         -0.4588      0.170       -2.693      0.007      -0.793     -0.125
## ma.L1          0.3663      0.171        2.138      0.033       0.031      0.702
## sigma2         0.0011    1.12e-05     94.399      0.000       0.001      0.001
## =====
## Ljung-Box (L1) (Q):                0.00    Jarque-Bera (JB):                53726.31
## Prob(Q):                          0.96    Prob(JB):                  0.00
## Heteroskedasticity (H):              1.60    Skew:                  -2.41
## Prob(H) (two-sided):                0.00    Kurtosis:                33.67
## =====
##
## Warnings:
## [1] Covariance matrix calculated using the outer product of gradients (complex-step).
```

```
model_21 = ARIMA(returns, order=(2, 0, 1)) # p=2, d=0, q=1
fit_21 = model_21.fit()
print(fit_21.summary())
```

```
##                                SARIMAX Results
## =====
## Dep. Variable:                ENR.DE.3    No. Observations:                1338
## Model:                      ARIMA(2, 0, 1)  Log Likelihood                2685.947
## Date:                      Fri, 27 Mar 2026  AIC                -5361.894
## Time:                      18:35:49      BIC                -5335.899
## Sample:                    0              HQIC                -5352.154
##                               - 1338
## Covariance Type:            opg
## =====
##              coef      std err          z      P>|z|      [0.025      0.975]
## -----
## const          0.0013      0.001        1.364      0.173      -0.001      0.003
## ar.L1         -0.8779      6.869       -0.128      0.898     -14.342     12.586
## ar.L2         -0.0755      0.681       -0.111      0.912      -1.411      1.260
```



```

## ma.L1          0.7802      6.870      0.114      0.910      -12.685      14.245
## sigma2         0.0011    1.13e-05    93.537      0.000      0.001      0.001
## =====
## Ljung-Box (L1) (Q):          0.01    Jarque-Bera (JB):          53545.78
## Prob(Q):          0.91    Prob(JB):          0.00
## Heteroskedasticity (H):      1.59    Skew:          -2.42
## Prob(H) (two-sided):        0.00    Kurtosis:          33.61
## =====
##
## Warnings:
## [1] Covariance matrix calculated using the outer product of gradients (complex-step).

```

Interpret the AR and MA coefficients. What do they say about short-term dependence in returns?

Let's first interpret the ARMA(1, 1) coefficients. From the results:

- $AR(1) = -0.4588$
- $MA(1) = 0.3663$
- $\sigma^2 \approx 0.001$

The **autoregressive term (AR)** measures how much yesterday's return influences today's return. Negative value implies mean-reversion. If yesterday's return was positive, today's return is slightly pulled downward and if yesterday's return was negative, today's return tends to move upwards. This indicates that returns slightly reverse previous day's movements. The **moving average (MA)** term measures how much yesterday's stock (unexpected return) affects today's return. Positive value implies shock persistence. If yesterday unexpected return was positive, today return will slightly increase and viceversa. This indicates that short-term surprises in returns have a small effect on the next day.

Conclusion:

- Negative $AR(1)$ -> returns tend to bounce back
- Positive $MA(1)$ -> short-lived stock persistence, sudden movements carry over to the next day.
- Daily log returns are weakly autocorrelated. Past values and past stocks have a small but measurable impact.

For ARMA(2, 1) coefficients:

- AR(1) = -0.8779 (very large standard error, not significant)
- AR(2) = -0.0755 (very large standard error, not significant)
- MA(1) = 0.7802 (very large standard error, not significant)
- σ^2 approx 0.001

The **autoregressive terms (AR1, AR2)** are not statistically significant, they have huge standard errors, so estimates are unreliable. Negative AR1 hints at mean-reversion, but due to insignificance we cannot confidently say returns show short-term dependence based on just AR terms. AR2 is close to zero -> little effect of returns from two years ago. The **moving average (MA1)** term is positive, but again not significant. This suggests that past stocks may have some short-lived effect, but we cannot rely on the estimate.

Conclusion:

- Model does not improve significantly over ARMA(1, 1). None of the additional parameters (AR2, MA1) are significant.
- Short term dependence is captured by the ARMA(1, 1) model.
- Adding a second AR term doesn't add meaningful predictions.

7.6. Forecast next 5 days of returns and visualize them vs historical returns

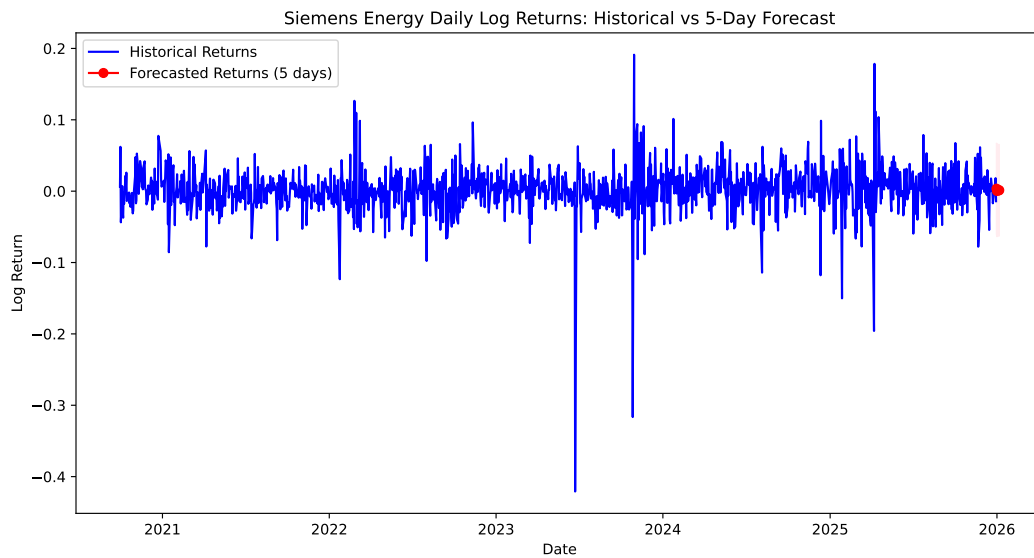
```
# forecast 5 days
forecast_steps = 5
forecast = fit_11.get_forecast(steps=forecast_steps)
forecast_mean = forecast.predicted_mean
forecast_ci = forecast.conf_int()

# returns.index is datetime
returns.index = pd.to_datetime(returns.index)

# create forecast index 5 business days
last_date = returns.index[-1]
forecast_index = pd.bdate_range(start=last_date + pd.Timedelta(days=1), periods=forecast_steps)

# plot historical and forecasted returns
plt.figure(figsize=(12,6))
plt.plot(returns.index, returns, label="Historical Returns", color='blue')
plt.plot(forecast_index, forecast_mean, label="Forecasted Returns (5 days)", color='red', marker='o')
plt.fill_between(forecast_index, forecast_ci.iloc[:,0], forecast_ci.iloc[:,1], color='pink', alpha=0.3)

plt.title("Siemens Energy Daily Log Returns: Historical vs 5-Day Forecast")
plt.xlabel("Date")
plt.ylabel("Log Return")
plt.legend()
plt.show()
```



7.7. Compare the performance of ARMA(1,1) vs ARMA(2,1) using AIC/BIC

```
# ARMA(1,1) summary
print("ARMA(1,1) AIC:", fit_11.aic)
```

```
## ARMA(1,1) AIC: -5365.703497389386
```

```
print("ARMA(1,1) BIC:", fit_11.bic)
```

```
## ARMA(1,1) BIC: -5344.9077724266335
```

```
# ARMA(2,1) summary
print("ARMA(2,1) AIC:", fit_21.aic)
```

```
## ARMA(2,1) AIC: -5361.893561616003
```

```
print("ARMA(2,1) BIC:", fit_21.bic)
```

```
## ARMA(2,1) BIC: -5335.898905412562
```

Interpretation:

- Both AIC and BIC are lower for ARMA(1, 1), meaning it fits the data better while being simpler.
- ARMA(2, 1) adds extra parameters that do not significantly improve the fit.
- Conclusion: ARMA(1, 1) is preferred over ARMA(2, 1) for Siemens Energy daily log returns; it captures short-term dependency without unnecessary complexity.

8. ARMA for weather data

8.1. Load the csv file and visualize the time series. Is there any visible trend or seasonality?

We use the free Open-Meteo Historical Weather API to download daily mean temperatures for **Barcelona, Spain** (lat 41.39, lon 2.15) for 2022–2025.

```
url = "https://archive-api.open-meteo.com/v1/archive"
params = {
    "latitude": 41.3888,
    "longitude": 2.1590,
    "start_date": "2020-01-01",
    "end_date": "2025-12-31",
    "daily": "temperature_2m_mean",
    "timezone": "Europe/Madrid",
}

response = requests.get(url, params=params, timeout=30)
response.raise_for_status()
data = response.json()

df = pd.DataFrame({
    "date": pd.to_datetime(data["daily"]["time"]),
    "temp": data["daily"]["temperature_2m_mean"],
})
df = df.dropna().reset_index(drop=True)
df = df.set_index("date")

print(f"Fetched {len(df)} daily observations: {df.index[0].date()} → {df.index[-1].date()}")

## Fetched 2192 daily observations: 2020-01-01 → 2025-12-31

print(df.describe().round(2))
```

```
##          temp
## count  2192.00
## mean    16.66
## std      6.18
## min      3.00
## 25%     11.40
## 50%     16.10
## 75%     22.20
## max     30.10
```

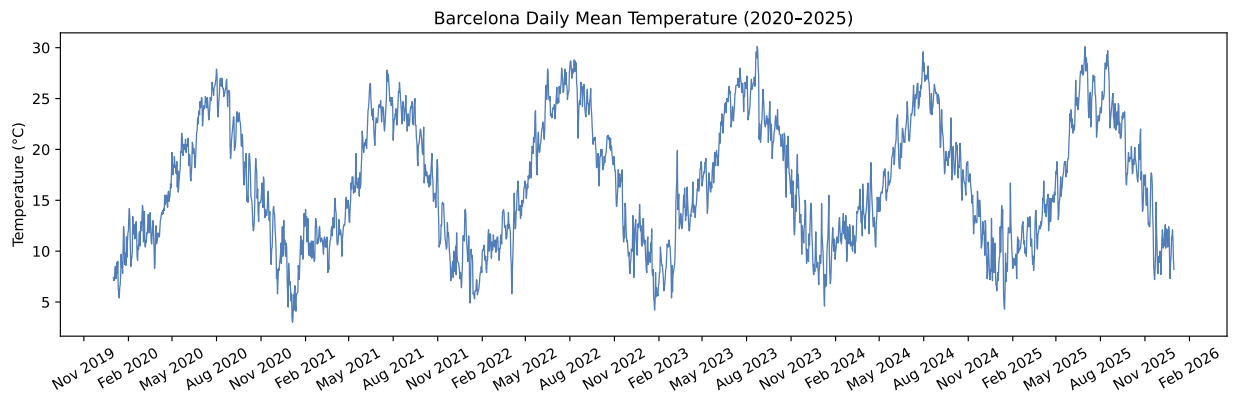
8.2. Visualize the raw time series

```
fig, ax = plt.subplots(figsize=(12, 4))
ax.plot(df.index, df["temp"], color="#3266ad", linewidth=0.9, alpha=0.85)
ax.set_title("Barcelona Daily Mean Temperature (2020-2025)")
ax.set_ylabel("Temperature (°C)")
ax.xaxis.set_major_formatter(mdates.DateFormatter("%b %Y"))
```

```
ax.xaxis.set_major_locator(mdates.MonthLocator(interval=3))
plt.xticks(rotation=30)
```

```
## (array([18201., 18293., 18383., 18475., 18567., 18659., 18748., 18840.,
##        18932., 19024., 19113., 19205., 19297., 19389., 19478., 19570.,
##        19662., 19754., 19844., 19936., 20028., 20120., 20209., 20301.,
##        20393., 20485.]), [Text(18201.0, 0, 'Nov 2019'), Text(18293.0, 0, 'Feb 2020'), Text(18383.0, 0, 'May 2020')])
```

```
plt.tight_layout()
plt.show()
```



Interpretation: We see a clear seasonality (aprox. 10 degrees in winter, aprox. 25 degrees in summer). There's no obvious long-term trend. The series is non-stationary due to seasonality.

8.3. Stationary test (ADF)

```
adf_result = adfuller(df["temp"], autolag="AIC")
print(f"ADF Statistic : {adf_result[0]}")
```

```
## ADF Statistic : -2.6082248378038373
```

```
print(f"p-value : {adf_result[1]}")
```

```
## p-value : 0.09130924801647272
```

```
print(f"Critical (5%) : {adf_result[4]['5%']}")
```

```
## Critical (5%) : -2.8628661117509484
```

Interpretation: The ADF test on the raw series shows $p = 0.091$, not stationary at 5% but borderline. This is expected because the series has seasonal non-stationarity (annual cycle), not just a trend.

8.4. Difference the series to achieve stationarity

We apply first-order differencing to remove the seasonal mean and make the series stationary.

```

diff = df["temp"].diff().dropna()

fig, axes = plt.subplots(2, 1, figsize=(12, 6), sharex=True)

axes[0].plot(df.index, df["temp"], color="#3266ad", linewidth=0.8)
axes[0].set_title("Original series")
axes[0].set_ylabel("Temp (°C)")

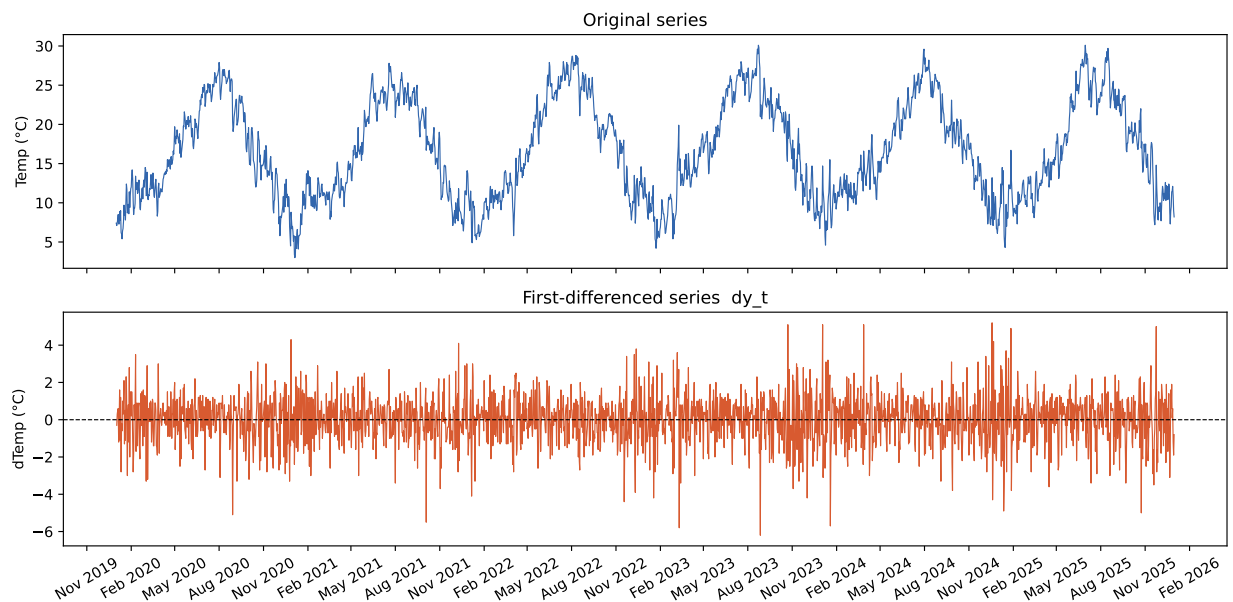
axes[1].plot(diff.index, diff.values, color="#D85A30", linewidth=0.8)
axes[1].axhline(0, color="black", linewidth=0.8, linestyle="--")
axes[1].set_title("First-differenced series dy_t")
axes[1].set_ylabel("dTemp (°C)")
axes[1].xaxis.set_major_formatter(mdates.DateFormatter("%b %Y"))
axes[1].xaxis.set_major_locator(mdates.MonthLocator(interval=3))

plt.xticks(rotation=30)

## (array([18201., 18293., 18383., 18475., 18567., 18659., 18748., 18840.,
##        18932., 19024., 19113., 19205., 19297., 19389., 19478., 19570.,
##        19662., 19754., 19844., 19936., 20028., 20120., 20209., 20301.,
##        20393., 20485.]), [Text(18201.0, 0, 'Nov 2019'), Text(18293.0, 0, 'Feb 2020'), Text(18383.0, 0, 'May 2020'),
##        Text(18475.0, 0, 'Aug 2020'), Text(18567.0, 0, 'Nov 2020'), Text(18659.0, 0, 'Feb 2021'), Text(18748.0, 0, 'May 2021'), Text(18840.0, 0, 'Aug 2021'),
##        Text(18932.0, 0, 'Nov 2021'), Text(19024.0, 0, 'Feb 2022'), Text(19113.0, 0, 'May 2022'), Text(19205.0, 0, 'Aug 2022'), Text(19297.0, 0, 'Nov 2022'), Text(19389.0, 0, 'Feb 2023'), Text(19478.0, 0, 'May 2023'), Text(19570.0, 0, 'Aug 2023'),
##        Text(19662.0, 0, 'Nov 2023'), Text(19754.0, 0, 'Feb 2024'), Text(19844.0, 0, 'May 2024'), Text(19936.0, 0, 'Aug 2024'), Text(20028.0, 0, 'Nov 2024'), Text(20120.0, 0, 'Feb 2025'), Text(20209.0, 0, 'May 2025'), Text(20301.0, 0, 'Aug 2025'),
##        Text(20393.0, 0, 'Nov 2025'), Text(20485.0, 0, 'Feb 2026')])

plt.tight_layout()
plt.show()

```



```

# ADF on differenced series
adf2 = adfuller(diff, autolag="AIC")
print(f"ADF on differenced: statistic={adf2[0]:.4f}, p-value={adf2[1]:.6f}")

```

```
## ADF on differenced: statistic=-20.3364, p-value=0.000000
```

We obtain a stationary series as p-value=0.000000

8.5. ACF and PACF of the Differenced Series

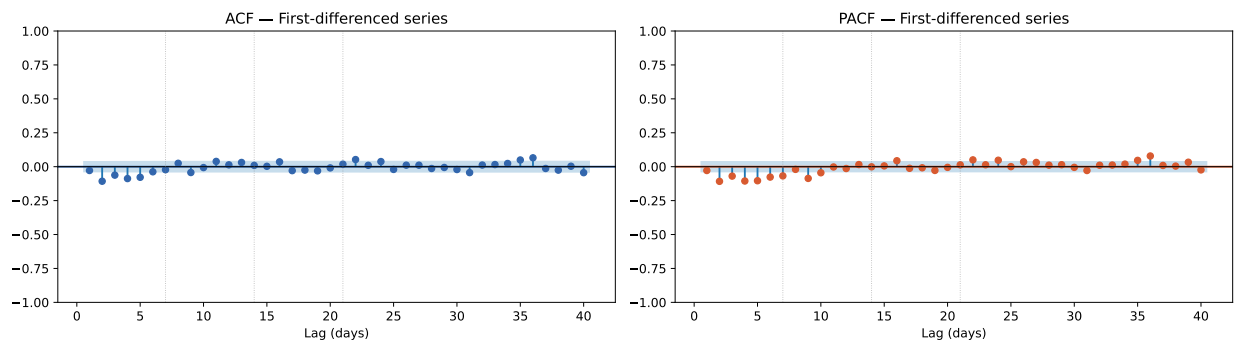
```
# Use only lag-1 differenced series (from the plot above)
diff = df["temp"].diff().dropna()

fig, axes = plt.subplots(1, 2, figsize=(14, 4))

plot_acf(diff, lags=40, ax=axes[0], color="#3266ad",
          title="ACF - First-differenced series", zero=False)
plot_pacf(diff, lags=40, ax=axes[1], color="#D85A30",
          title="PACF - First-differenced series", zero=False, method="ywm")

for ax in axes:
    ax.set_xlabel("Lag (days)")
    ax.axhline(0, color="black", linewidth=0.5)
    # Add reference lines at lag 7 and 365 to spot weekly/annual patterns
    for lag in [7, 14, 21]:
        ax.axvline(lag, color="gray", linestyle=":", linewidth=0.7, alpha=0.5)

plt.tight_layout()
plt.show()
```



Nearly all lags within the confidence bands. The first-differenced Barcelona temperature series behaves like random walk with noise. Daily temperatures changes are unpredictable from past changes alone. This is realistic, as whether tomorrow is 1 degree warmer or cooler than today carries almost no information about the day after. The only signal perceived is a small negative cluster at lags 1-5 in the PACF (values around -0.10 to -0.15), which barely touched the band edge. This is the only exploitable structure from the plots.

Model selection: we will select ARIMA (1, 1, 1). We don't want to overfit with AR(2) or higher.

8.6. Fit ARIMA(1, 1, 1) Model

```
model = ARIMA(df["temp"], order=(1, 1, 1))
result = model.fit()
print(result.summary())
```

```
##                                SARIMAX Results
## =====
## Dep. Variable:                  temp    No. Observations:                2192
```

```

## Model:                ARIMA(1, 1, 1)    Log Likelihood        -3645.762
## Date:                  Fri, 27 Mar 2026    AIC                    7297.524
## Time:                  18:35:50          BIC                    7314.600
## Sample:                01-01-2020        HQIC                   7303.765
##                        - 12-31-2025
## Covariance Type:      opg
## =====
##              coef      std err          z      P>|z|      [0.025      0.975]
## -----
## ar.L1           0.7299      0.034      21.284      0.000      0.663      0.797
## ma.L1          -0.8674      0.026     -32.869      0.000     -0.919     -0.816
## sigma2          1.6323      0.039      41.868      0.000      1.556      1.709
## =====
## Ljung-Box (L1) (Q):                9.60    Jarque-Bera (JB):                174.05
## Prob(Q):                          0.00    Prob(JB):                      0.00
## Heteroskedasticity (H):              1.26    Skew:                      -0.24
## Prob(H) (two-sided):                0.00    Kurtosis:                     4.30
## =====
##
## Warnings:
## [1] Covariance matrix calculated using the outer product of gradients (complex-step).

```

```

phi1  = result.arparams[0]
theta1 = result.maparams[0]
sigma2 = result.params["sigma2"]
print(f"\n    = {phi1:.4f}    (AR lag-1)")

```

```

##
##    = 0.7299    (AR lag-1)

```

```

print(f"    = {theta1:.4f}    (MA lag-1)")

```

```

##    = -0.8674    (MA lag-1)

```

```

print(f"  ^2 = {sigma2:.4f}")

```

```

##  ^2 = 1.6323

```

```

# residual diagnostics plot
resid = result.resid.dropna()

fig, axes = plt.subplots(1, 3, figsize=(15, 4))

# residuals over time
axes[0].plot(resid.index, resid.values, color="#888780", linewidth=0.6)
axes[0].axhline(0, color="black", linewidth=0.8, linestyle="--")
axes[0].set_title("Residuals over time")
axes[0].set_ylabel("_t (°C)")

# distribution
axes[1].hist(resid, bins=50, color="#3266ad", edgecolor="white", linewidth=0.3)

```



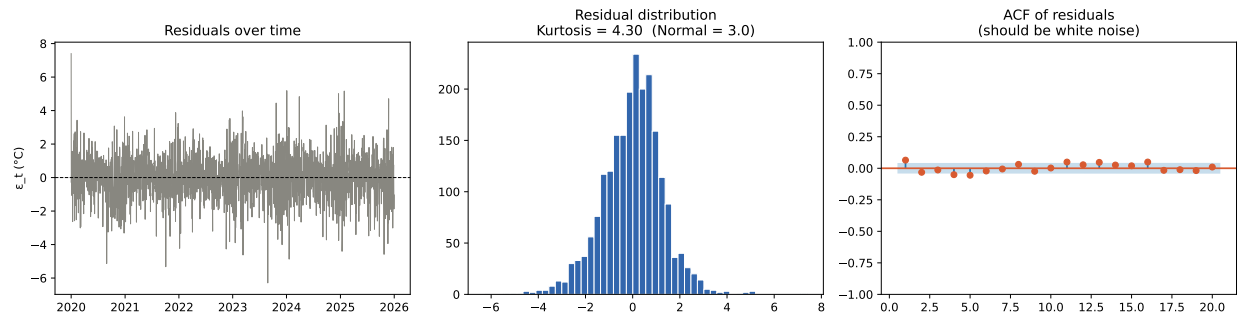
```

axes[1].set_title(f"Residual distribution\nKurtosis = 4.30 (Normal = 3.0)")

# ACF of residuals
from statsmodels.graphics.tsaplots import plot_acf
plot_acf(resid, lags=20, ax=axes[2], color="#D85A30",
         title="ACF of residuals\n(should be white noise)", zero=False)

plt.tight_layout()
plt.show()

```



The model is well specified, the only real issue is non-gaussianity...

8.7. Forecast next 7 days of temperature changes

```

steps = 7 # days
forecast = result.get_forecast(steps=steps)
fc_level = forecast.predicted_mean # temperature level
fc_ci = forecast.conf_int(alpha=0.05)

last_date = df.index[-1]
last_temp = df["temp"].iloc[-1]
fc_dates = pd.date_range(last_date + pd.Timedelta(days=1), periods=steps, freq="D")
fc_level.index = fc_dates
fc_ci.index = fc_dates

prev_levels = np.concatenate([[last_temp], fc_level.values[:-1]])
delta_hat = fc_level.values - prev_levels

```

8.8. Convert the predicted changes back to temperature predictions + visualize

```

cumulative = np.cumsum(delta_hat)
manual_temps = last_temp + cumulative # should match fc_level.values exactly
print(np.round(manual_temps - fc_level.values, 6)) # should be all aprox 0

## [0. 0. 0. 0. 0. 0. 0.]

```

8.8. Plot actual vs fitted values for the last 30 days

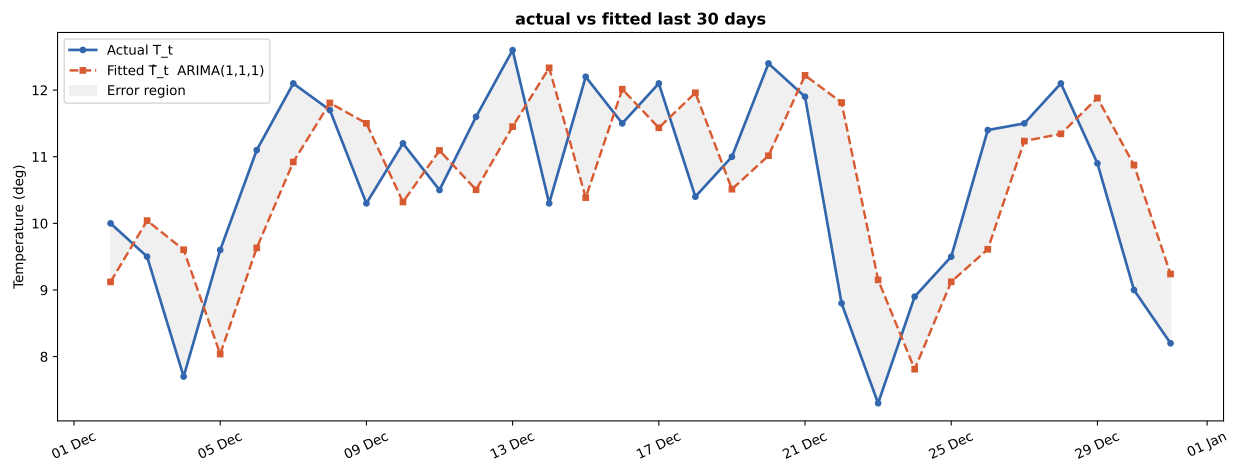
```
fitted_vals = result.fittedvalues
last30_actual = df["temp"].iloc[-30:]
last30_fitted = fitted_vals.loc[last30_actual.index]
residuals_30 = last30_actual - last30_fitted

fig, ax = plt.subplots(figsize=(13, 5))

ax.plot(last30_actual.index, last30_actual.values,
        color="#3266ad", linewidth=2, marker="o",
        markersize=4, label="Actual T_t")
ax.plot(last30_fitted.index, last30_fitted.values,
        color="#D85A30", linewidth=1.8, linestyle="--",
        marker="s", markersize=4, label="Fitted T_t ARIMA(1,1,1)")
ax.fill_between(last30_actual.index,
               last30_actual.values, last30_fitted.values,
               alpha=0.12, color="#888780", label="Error region")
ax.set_title("actual vs fitted last 30 days", fontweight="bold")
ax.set_ylabel("Temperature (deg)")
ax.legend()
ax.xaxis.set_major_formatter(mdates.DateFormatter("%d %b"))
plt.xticks(rotation=25)
```

```
## (array([20423., 20427., 20431., 20435., 20439., 20443., 20447., 20451.,
##        20454.]), [Text(20423.0, 0, '01 Dec'), Text(20427.0, 0, '05 Dec'), Text(20431.0, 0, '09 Dec')])
```

```
plt.tight_layout()
plt.show()
```



```
rmse = np.sqrt(np.mean(residuals_30**2))
mae = np.mean(np.abs(residuals_30))
print(f"RMSE : {rmse:.3f} degrees")
```

```
## RMSE : 1.315 degrees
```

```
print(f"MAE : {mae:.3f} degrees")
```

```
## MAE : 1.146 degrees
```

The fitted line tracks the actual one with a one-day lag. The model always knows where temperature was yesterday (AR) and corrects for yesterday's error (MA). It follows the shape well but misses turning points by one step. Limitation of ARIMA model: cannot anticipate direction changes, only follow them.

The gray area (error) is narrow most of the time, widening on 04 Dec and 24 Dec, corresponding to extreme days that caused the leptokurtic residual distribution saw in previous plots.

8.9. Optional: try using a seasonal ARIMA (SARIMA) if there is weekly or yearly seasonality in temperature

Daily temperature has annual seasonality (summer/winter cycle), not weekly. The seasonal period we will use is $m=365$.

```
sarima = SARIMAX(
    df["temp"],
    order=(1, 1, 1),
    seasonal_order=(1, 0, 0, 365),
    enforce_stationarity=False,
    enforce_invertibility=False,
)
sarima_result = sarima.fit(dispatch=False)
print(sarima_result.summary())
```

```
##                                SARIMAX Results
## =====
## Dep. Variable:                temp    No. Observations:                2192
## Model:                      SARIMAX(1, 1, 1)x(1, 0, [], 365)  Log Likelihood                -3046.770
## Date:                      Fri, 27 Mar 2026    AIC                6101.540
## Time:                      18:37:31    BIC                6123.577
## Sample:                    01-01-2020    HQIC                6109.669
##                               - 12-31-2025
## Covariance Type:                opg
## =====
##              coef      std err          z      P>|z|      [0.025      0.975]
## -----
## ar.L1          0.7271      0.037     19.663      0.000      0.655      0.800
## ma.L1         -0.8666      0.029    -30.218      0.000     -0.923     -0.810
## ar.S.L365       0.0202      0.024      0.856      0.392     -0.026      0.067
## sigma2         1.6492      0.043     38.457      0.000      1.565      1.733
## =====
## Ljung-Box (L1) (Q):                6.99    Jarque-Bera (JB):                154.60
## Prob(Q):                          0.01    Prob(JB):                          0.00
## Heteroskedasticity (H):            1.31    Skew:                          -0.20
## Prob(H) (two-sided):              0.00    Kurtosis:                         4.37
## =====
##
## Warnings:
## [1] Covariance matrix calculated using the outer product of gradients (complex-step).
```

Why (1, 0, 0)[365] seasonal part? - P=1 -> AR at lag 365: today correlates with same day last year - D=0 -> no seasonal differencing (d=1 already handles stationarity) - Q=0 -> no seasonal MA

Parameter interpretation:

```
phi1_s = sarima_result.arparams[0]      # non-seasonal AR(1)
theta1_s = sarima_result.maparams[0]    # non-seasonal MA(1)
Phi1 = sarima_result.seasonalarparams[0] # seasonal AR(1) at lag 365
print(f"\nNon-seasonal      = {phi1_s:.4f}")
```

```
##
## Non-seasonal      = 0.7271
```

```
print(f"Non-seasonal      = {theta1_s:.4f}")
```

```
## Non-seasonal      = -0.8666
```

```
print(f"Seasonal      Φ = {Phi1:.4f}")
```

```
## Seasonal      Φ = 0.0202
```

- = 0.7271 -> today's temperature change is 73% correlated with yesterday's
- = -0.8666 -> the model rapidly undoes yesterday's forecast mistake
- $\Phi = 0.0202$ -> the seasonal correction from 365 days ago contributes only 2% weight... Practically zero.

This means that the 7-day forecast from SARIMA will be almost identical to the plain ARIMA one, because the seasonal term contributes the most.

Model comparison:

```
print(f"ARIMA(1,1,1): AIC: {result.aic:.1f}, BIC: {result.bic:.1f}")
```

```
## ARIMA(1,1,1): AIC: 7297.5, BIC: 7314.6
```

```
print(f"SARIMA(1,1,1)(1,0,0)[365]: AIC: {sarima_result.aic:.1f}, BIC: {sarima_result.bic:.1f}")
```

```
## SARIMA(1,1,1)(1,0,0)[365]: AIC: 6101.5, BIC: 6123.6
```

```
delta_aic = result.aic - sarima_result.aic
```

Fitted vs actual last 30 days: ARIMA vs SARIMA

```
sarima_fitted = sarima_result.fittedvalues
s_last30 = sarima_fitted.loc[last30_actual.index]
s_residuals_30 = last30_actual - s_last30

fig, ax = plt.subplots(figsize=(13, 5))

ax.plot(last30_actual.index, last30_actual.values,
```

```

        color="#3266ad", linewidth=2, marker="o",
        markersize=4, label="Actual T_t")
ax.plot(last30_fitted.index, last30_fitted.values,
        color="#D85A30", linewidth=1.6, linestyle="--",
        marker="s", markersize=3, label="ARIMA(1,1,1) fitted", alpha=0.8)
ax.plot(s_last30.index, s_last30.values,
        color="#639922", linewidth=1.6, linestyle="-. ",
        marker="^", markersize=4, label="SARIMA(1,1,1)(1,0,0)[365] fitted")

ax.set_title("Actual vs Fitted last 30 days\nARIMA vs SARIMA")
ax.set_ylabel("Temperature (deg)")
ax.legend()
ax.xaxis.set_major_formatter(mdates.DateFormatter("%d %b"))
plt.xticks(rotation=25)

```

```

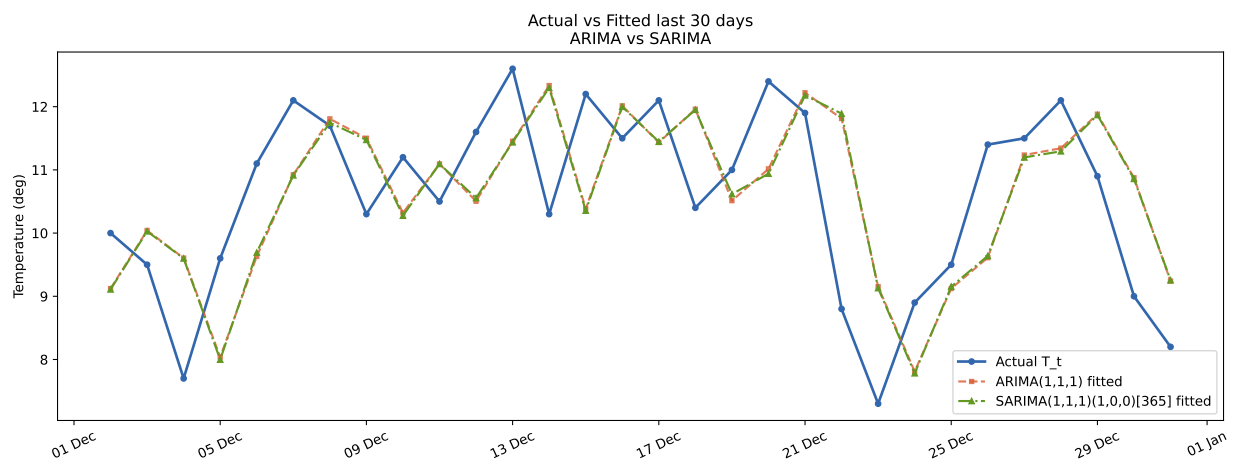
## (array([20423., 20427., 20431., 20435., 20439., 20443., 20447., 20451.,
##         20454.]), [Text(20423.0, 0, '01 Dec'), Text(20427.0, 0, '05 Dec'), Text(20431.0, 0, '09 Dec')])

```

```

plt.tight_layout()
plt.show()

```



```

# Metrics comparison

```

```

rmse_s = np.sqrt(np.mean(s_residuals_30**2))
mae_s = np.mean(np.abs(s_residuals_30))
print(f"\n Last-30-day comparison")

```

```

##
## Last-30-day comparison

```

```

print(f"ARIMA RMSE: {rmse:.3f} degrees MAE: {mae:.3f} degrees")

```

```

## ARIMA RMSE: 1.315 degrees MAE: 1.146 degrees

```

```
print(f"SARIMA RMSE: {rmse_s:.3f} degrees    MAE: {mae_s:.3f} degrees")
```

```
## SARIMA RMSE: 1.318 degrees    MAE: 1.142 degrees
```

```
print(f"RMSE improvement: {rmse - rmse_s:.3f} degrees")
```

```
## RMSE improvement: -0.003 degrees
```